

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ



BÀI TẬP LỚN

THIẾT KẾ VLSI

Đề tài:

**THIẾT KẾ VÀ TỔNG HỢP HỆ THỐNG GIẢI MÃ
VITERBI
TỪ RTL ĐẾN GDSII SỬ DỤNG OPENLANE FLOW**

Giảng viên hướng dẫn: TS. Nguyễn Vũ Thắng
Sinh viên thực hiện: Phạm Chí Dũng – 20200106
Võ Ngọc Vinh – 20227447
Nguyễn Văn Dương – 20241713E
Lớp: 163187 – ET4340

Hà Nội, 2026

TÓM TẮT NỘI DUNG

Báo cáo này trình bày quy trình toàn diện về thiết kế, kiểm chứng và thực thi vật lý cho một hệ thống giải mã Viterbi (Viterbi Decoder SoC). Hệ thống được xây dựng hoàn chỉnh với các khối giao tiếp ngoại vi (Sync FIFO, PISO, SIPO) để xử lý dữ liệu dòng. Điểm nổi bật của thiết kế là việc áp dụng kiến trúc **Register Exchange (RE)** trong đơn vị truy vết, giúp tối ưu hóa băng thông xử lý và giảm độ trễ so với phương pháp Traceback truyền thống.

Kết quả kiểm tra chức năng (Functional Verification) cho thấy mạch hoạt động chính xác tuyệt đối với bộ dữ liệu kiểm thử. Quy trình thiết kế vật lý (Physical Design) được thực hiện bằng công cụ mã nguồn mở **OpenLane Flow** với công nghệ Sky130, đạt kết quả khả quan về diện tích ($0.16mm^2$), công suất tiêu thụ thấp và đáp ứng tốt các yêu cầu về thời gian (Timing) ở tần số 80MHz.

Mục lục

TÓM TẮT NỘI DUNG	1
1 TỔNG QUAN VỀ MÃ TÍCH CHẬP VÀ GIẢI MÃ VITERBI	7
1.1 Giới thiệu mã tích chập	7
1.2 Nguyên lý thiết kế Viterbi (Design Architecture)	7
2 ĐẶC TẢ KỸ THUẬT VÀ THIẾT KẾ RTL	9
2.1 Đặc tả kỹ thuật (Specifications)	9
2.1.1 Mục tiêu thiết kế	9
2.2 Kiến trúc hệ thống và Chi tiết I/O từng khối	10
2.2.1 Khối đệm dữ liệu (Sync FIFO)	10
2.2.2 Khối chuyển đổi song song sang nối tiếp (PISO)	11
2.2.3 Khối tính toán chi phí nhánh (BMU)	12
2.2.4 Khối Cộng - So sánh - Chọn (ACSU)	13
2.2.5 Khối quản lý Path Metric (PMU)	15
2.2.6 Khối truy vết Survivor Memory (TBU - Register Exchange)	15
2.2.7 Khối chuyển đổi nối tiếp sang song song (SIPO)	16
2.3 Triển khai RTL (RTL Implementation)	17
3 KIỂM TRA CHỨC NĂNG MẠCH	20
3.1 Môi trường kiểm chứng (Testbench Environment)	20
3.2 Kết quả mô phỏng (Simulation Results)	20
3.2.1 Khối đệm dữ liệu (Sync FIFO)	20
3.2.2 Khối chuyển đổi song song sang nối tiếp (PISO)	22
3.2.3 Khối tính toán nhánh (BMU)	24
3.2.4 Khối Cộng - So sánh - Chọn (ACSU)	26
3.2.5 Khối quản lý Metric (PMU)	28
3.2.6 Khối truy vết (TBU)	30
3.2.7 Khối chuyển đổi nối tiếp sang song song (SIPO)	32
3.2.8 Kiểm chứng toàn hệ thống (System Integration)	34
4 TỔNG HỢP VÀ THIẾT KẾ VẬT LÝ	38
4.1 Chiến lược thiết kế và Quy trình OpenLane	38
4.1.1 Chiến lược thiết kế phân cấp (Hierarchical Design)	38
4.1.2 Quy trình thực hiện trên OpenLane	38
4.2 Thực thi Hardening các khối Macro	39
4.2.1 Thiết lập cấu hình (Configuration)	39

4.2.2	Kết quả Hardening	40
4.3	Tích hợp hệ thống (Top-level Integration)	40
4.3.1	Sắp xếp Macro (Macro Placement)	41
4.3.2	Kết quả thiết kế vật lý chi tiết	41
KẾT LUẬN		44

Danh sách hình vẽ

2.1	Sơ đồ khối Top-Level System	10
2.2	Sơ đồ khối Sync FIFO	11
2.3	Sơ đồ khối PISO	12
2.4	Sơ đồ khối BMU	13
2.5	Sơ đồ khối ACSU	14
2.6	Sơ đồ khối PMU	15
2.7	Sơ đồ khối TBU	16
2.8	Sơ đồ khối SIPO	17
2.9	Sơ đồ khối chi tiết hệ thống Viterbi Decoder SoC	18
2.10	Lưu đồ thuật toán điều khiển luồng dữ liệu hệ thống	18
3.1	Log mô phỏng Sync FIFO - Phần 1	21
3.2	Log mô phỏng Sync FIFO - Phần 2	21
3.3	Giản đồ sóng tín hiệu Sync FIFO	22
3.4	Log mô phỏng PISO - Phần 1	23
3.5	Log mô phỏng PISO - Phần 2	23
3.6	Giản đồ sóng tín hiệu PISO	24
3.7	Log mô phỏng BMU - Phần 1	25
3.8	Log mô phỏng BMU - Phần 2	25
3.9	Giản đồ sóng tín hiệu BMU	26
3.10	Log mô phỏng ACSU - Phần 1	27
3.11	Log mô phỏng ACSU - Phần 2	27
3.12	Giản đồ sóng tín hiệu ACSU	28
3.13	Log mô phỏng PMU - Phần 1	29
3.14	Log mô phỏng PMU - Phần 2	29
3.15	Giản đồ sóng tín hiệu PMU	30
3.16	Log mô phỏng TBU - Phần 1	31
3.17	Log mô phỏng TBU - Phần 2	31
3.18	Giản đồ sóng tín hiệu TBU	32
3.19	Log mô phỏng SIPO - Phần 1	33
3.20	Log mô phỏng SIPO - Phần 2	33
3.21	Giản đồ sóng tín hiệu SIPO	34
3.22	Bảng tổng kết (Test Summary Report) từ Simulation Log	36
3.23	[SYS_01] Sanity Check: Hệ thống giải mã chính xác dữ liệu chuẩn	36
3.24	[SYS_03] Single Bit Error: Hệ thống tự động sửa sai khi đầu vào bị lỗi 1 bit	37
3.25	[SYS_07] Continuous Stress: Hoạt động ổn định dưới tải liên tục	37

4.1	Layout các khối tính toán Metric	42
4.2	Layout khối xử lý trung tâm	43
4.3	Layout các khối giao tiếp dữ liệu	43
4.4	Layout toàn hệ thống Viterbi Decoder SoC (Final GDSII)	43

Danh sách bảng

2.1	Các thông số thiết kế chính	9
2.2	Mục tiêu thiết kế hệ thống Viterbi Decoder	9
2.3	Đặc tả giao diện System Top	10
2.4	Giao diện khối Sync FIFO	11
2.5	Giao diện khối PISO	12
2.6	Giao diện khối BMU	13
2.7	Giao diện khối ACSU	14
2.8	Giao diện khối PMU	15
2.9	Giao diện khối TBU	16
2.10	Giao diện khối SIPO	17
3.1	Các kịch bản kiểm thử khối Sync FIFO	20
3.2	Các kịch bản kiểm thử khối PISO	22
3.3	Các kịch bản kiểm thử khối BMU	24
3.4	Các kịch bản kiểm thử khối ACSU	26
3.5	Các kịch bản kiểm thử khối PMU	28
3.6	Các kịch bản kiểm thử khối TBU	30
3.7	Các kịch bản kiểm thử khối SIPO	32
3.8	Tổng hợp kết quả các kịch bản kiểm thử hệ thống	35
4.1	Tổng hợp kết quả tổng hợp vật lý (OpenLane Metrics)	42

Chương 1: TỔNG QUAN VỀ MÃ TÍCH CHẬP VÀ GIẢI MÃ VITERBI

1.1 Giới thiệu mã tích chập

Mã tích chập (Convolutional Codes) là một lớp mã sửa lỗi quan trọng, trong đó luồng bit đầu ra được tạo ra từ việc thực hiện phép tích chập giữa luồng bit đầu vào với đáp ứng xung của các thanh ghi dịch. Một bộ mã hóa tích chập được đặc trưng bởi bộ tham số (n, k, m) , trong đó:

- k : Số bit đầu vào tại mỗi nhịp thời gian.
- n : Số bit đầu ra tại mỗi nhịp thời gian.
- m : Độ sâu của bộ nhớ (số lượng thanh ghi dịch).

Độ dài giới hạn quan trọng $K = m + 1$ biểu thị số lượng bit đầu vào ảnh hưởng đến một bit đầu ra. Tỷ lệ mã $R = k/n$ biểu thị hiệu suất băng thông.

1.2 Nguyên lý thiết kế Viterbi (Design Architecture)

Kiến trúc phần cứng của bộ giải mã được thiết kế dựa trên thuật toán Viterbi tiêu chuẩn nhưng được tối ưu hóa cho tốc độ xử lý và khả năng hiện thực hóa trên phần cứng (Hardware Implementation). Thiết kế bao gồm ba thành phần dữ liệu chính hoạt động theo cơ chế pipeline:

1. **Branch Metric Unit (BMU) - Hard Decision:** Khối BMU thực hiện tính toán khoảng cách Hamming (Hamming Distance) giữa cặp bit đầu vào (từ kênh truyền) và các cặp bit lý thuyết của mã lưới. Do sử dụng quyết định cứng (Hard Decision), metric nhánh chỉ đơn giản là số lượng bit sai khác (0, 1 hoặc 2) thay vì khoảng cách Euclid phức tạp. Tất cả các metric nhánh được tính toán song song trong cùng một chu kỳ.
2. **Add-Compare-Select Unit (ACSU) - Parallel Update:** Đây là trái tim của bộ giải mã. Tại mỗi thời điểm, ACSU cập nhật chi phí đường dẫn (Path Metric) cho tất cả 4 trạng thái (S_0, S_1, S_2, S_3) đồng thời.
 - **Add:** Cộng Path Metric hiện tại với Branch Metric tương ứng.
 - **Compare:** So sánh hai đường dẫn đi tới cùng một trạng thái.
 - **Select:** Chọn đường dẫn có chi phí nhỏ hơn (Minimum Metric) làm đường tồn tại (Survivor Path) và cập nhật lại Path Metric mới.

3. **Traceback Unit (TBU) - Register Exchange Architecture:** Thay vì sử dụng phương pháp Traceback truyền thống (lưu vết vào RAM và truy xuất ngược) gây độ trễ lớn và thông lượng thấp, đề án áp dụng kiến trúc **Register Exchange (RE)**.

- Mỗi trạng thái quản lý một thanh ghi dịch lịch sử (History Register) có độ dài $L = 15$ (Traceback Length).
- Tại mỗi nhịp clock, dựa trên bit quyết định (Decision Bit) từ ACSU, nội dung của các thanh ghi lịch sử được hoán đổi và cập nhật dòng dữ liệu.
- Bit giải mã đầu ra được chọn từ thanh ghi lịch sử của trạng thái có Path Metric nhỏ nhất (Most Likelihood State), đảm bảo độ tin cậy cao nhất.

Chương 2: ĐẶC TẢ KỸ THUẬT VÀ THIẾT KẾ RTL

2.1 Đặc tả kỹ thuật (Specifications)

Dựa trên yêu cầu của đề tài, hệ thống Viterbi Decoder được thiết kế với các thông số kỹ thuật sau:

Bảng 2.1: Các thông số thiết kế chính

Tham số	Giá trị
Constraint Length (K)	3
Code Rate (R)	1/2
Generators (Octal)	$G_1 = 7_8(111_2)$, $G_2 = 5_8(101_2)$
Traceback Length (L)	15
Architecture Type	Register Exchange (RE)
Soft/Hard Decision	Hard Decision (1-bit quantization)

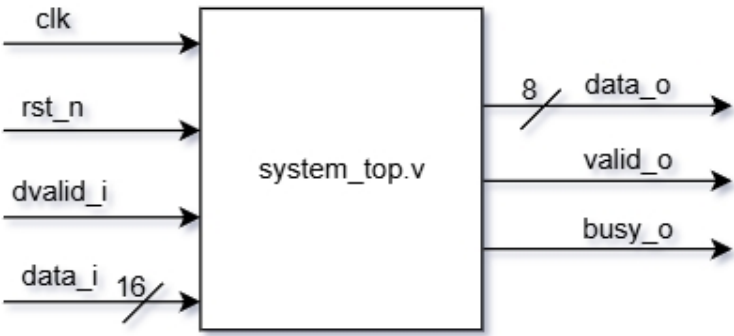
2.1.1 Mục tiêu thiết kế

Dựa trên yêu cầu ứng dụng và giới hạn của công nghệ Sky130, các mục tiêu thiết kế cụ thể được đặt ra như sau:

Bảng 2.2: Mục tiêu thiết kế hệ thống Viterbi Decoder

Tiêu chí	Mục tiêu	Ghi chú
Tần số hoạt động tối đa	≥ 80 MHz	Đảm bảo throughput 80 Mbps với 1-bit/cycle.
Diện tích chip (Die Area)	≤ 4 mm ²	Tối ưu cho ứng dụng IoT/nhúng.
Công suất tiêu thụ	≤ 10 mW	Phù hợp thiết bị di động.
Timing Closure	0 Violations	Đạt Setup/Hold tại tần số mục tiêu.
Physical Verification	Pass DRC/LVS	Sẵn sàng cho Tape-out.
Kiến trúc xử lý	Pipeline, 1-bit/cycle	Xử lý dòng bit liên tục.
Độ trễ giải mã (Latency)	≤ 20 cycles	Từ lúc nhận bit đến khi xuất kết quả.

Bảng tín hiệu giao tiếp của khối Top-Level (system_top):



Hình 2.1: Sơ đồ khối Top-Level System

Bảng 2.3: Đặc tả giao diện System Top

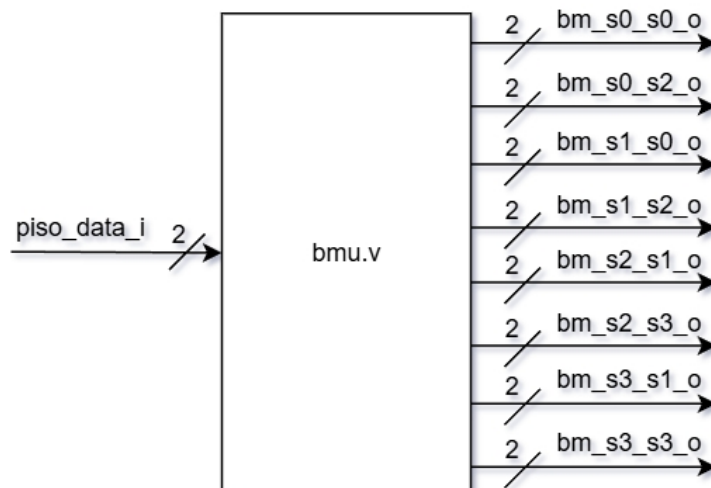
Tên tín hiệu	Độ rộng	Hướng	Mô tả
clk	1	I	Xung clock hệ thống (Cạnh dương).
rst_n	1	I	Reset hệ thống (Tích cực thấp).
dvalid_i	1	I	Tín hiệu báo dữ liệu đầu vào hợp lệ (Sync FIFO WrEn).
data_i	16	I	Gói dữ liệu đầu vào (16-bit).
data_o	8	O	Dữ liệu byte đã giải mã (8-bit).
valid_o	1	O	Tín hiệu báo dữ liệu đầu ra đã sẵn sàng.
busy_o	1	O	Báo bận khi bộ đệm Sync FIFO đầy.

2.2 Kiến trúc hệ thống và Chi tiết I/O từng khối

Hệ thống được thiết kế theo luồng dữ liệu Pipeline để tối ưu hóa hiệu năng. Dưới đây là mô tả chi tiết các chân tín hiệu của từng thực thể chức năng.

2.2.1 Khối đệm dữ liệu (Sync FIFO)

Khối này lưu trữ tạm thời dữ liệu 16-bit từ bên ngoài trước khi đưa vào bộ giải mã.



Hình 2.2: Sơ đồ khối Sync FIFO

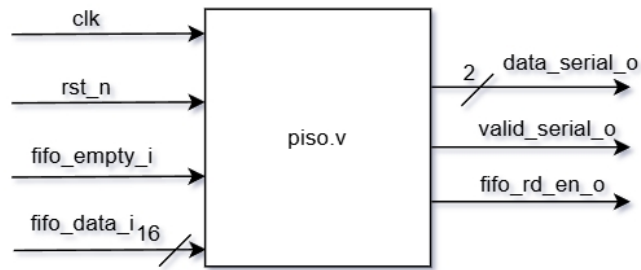
Mô tả luồng hoạt động: Bộ đệm Sync FIFO hoạt động như một bộ nhớ vòng (Circular Buffer) với độ sâu 16 phần tử. Khi tín hiệu ghi (wr_en_i) tích cực, dữ liệu đầu vào được lưu vào địa chỉ của con trỏ ghi (wr_ptr). Khi có yêu cầu đọc (rd_en_i), dữ liệu được lấy ra từ địa chỉ của con trỏ đọc (rd_ptr). Bộ đếm ($count$) theo dõi số lượng phần tử hiện có để tạo ra các cờ trạng thái $full_o$ và $empty_o$, giúp ngăn chặn hiện tượng tràn (overflow) hoặc đọc rỗng (underflow).

Bảng 2.4: Giao diện khối Sync FIFO

Tên tín hiệu	Độ rộng	Hướng	Mô tả
clk	1	I	Xung clock hệ thống.
rst_n	1	I	Reset hệ thống (Tích cực thấp).
wr_en_i	1	I	Lệnh ghi dữ liệu vào Sync FIFO.
wr_data_i	16	I	Dữ liệu 16-bit ghi vào.
$full_o$	1	O	Cờ báo bộ đệm đã đầy.
rd_en_i	1	I	Lệnh đọc dữ liệu ra (từ PISO).
rd_data_o	16	O	Dữ liệu 16-bit đọc ra.
$empty_o$	1	O	Cờ báo bộ đệm rỗng.

2.2.2 Khối chuyển đổi song song sang nối tiếp (PISO)

Chuyển đổi gói 16-bit thành các cặp 2-bit (symbol) để nạp vào Viterbi Core.



Hình 2.3: Sơ đồ khối PISO

Mô tả luồng hoạt động: PISO được điều khiển bởi một máy trạng thái (FSM) gồm 3 trạng thái:

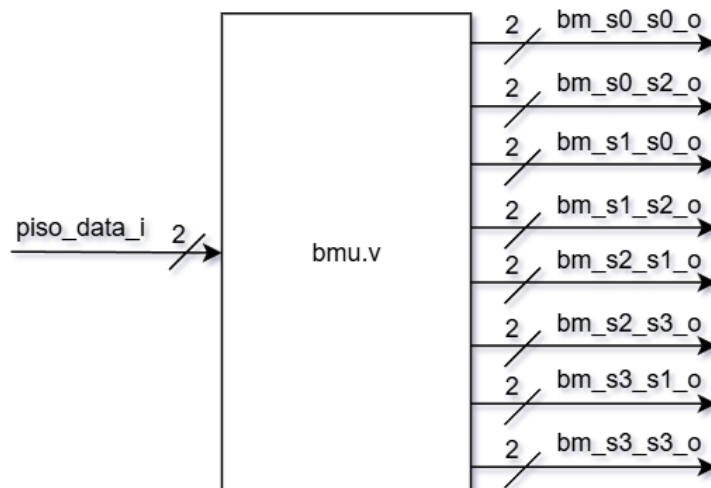
- **IDLE:** Chờ FIFO báo có dữ liệu (!`fifo_empty_i`). Khi có dữ liệu, chuyển sang chu kỳ đọc.
- **READ_WAIT:** Kích hoạt tín hiệu đọc gửi tới FIFO (`fifo_rd_en_o`), nạp 16-bit dữ liệu vào thanh ghi dịch nội bộ.
- **SHIFT:** Thực hiện dịch trái thanh ghi 8 lần. Tại mỗi nhịp, 2 bit cao nhất (MSB) được đẩy ra ngoài (`data_serial_o`) để cung cấp cho Viterbi Core.

Bảng 2.5: Giao diện khối PISO

Tên tín hiệu	Độ rộng	Hướng	Mô tả
clk	1	I	Xung clock hệ thống.
rst_n	1	I	Reset hệ thống (Tích cực thấp).
fifo_data_i	16	I	Dữ liệu nhận từ Sync FIFO.
fifo_empty_i	1	I	Trạng thái rỗng của Sync FIFO.
fifo_rd_en_o	1	O	Lệnh yêu cầu Sync FIFO xuất dữ liệu.
data_serial_o	2	O	Cặp bit symbol gửi tới Core.
valid_serial_o	1	O	Báo symbol hiện tại hợp lệ.

2.2.3 Khối tính toán chi phí nhánh (BMU)

Tính khoảng cách Hamming giữa cặp bit nhận được và các nhánh kỳ vọng trong đồ thị lưới.



Hình 2.4: Sơ đồ khối BMU

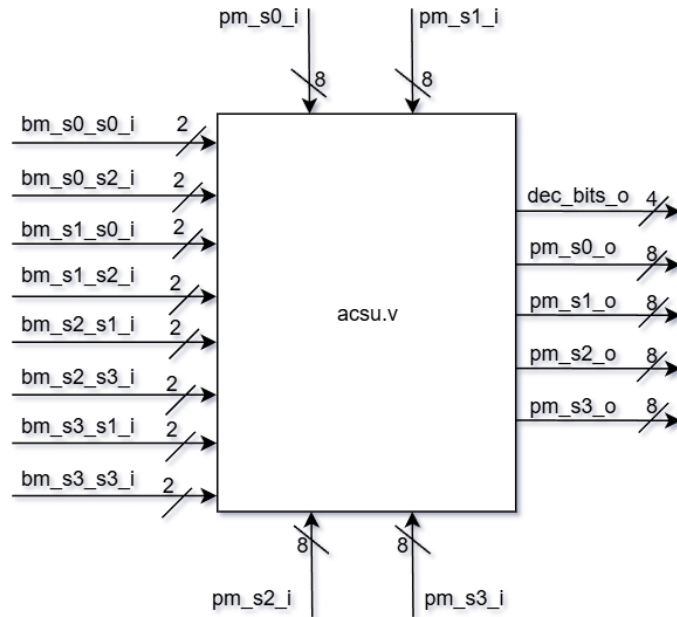
Mô tả luồng hoạt động: Khối BMU nhận đầu vào là cặp bit symbol từ PISO và so sánh với 4 cặp nhánh lý thuyết (00, 01, 10, 11). Hàm tính khoảng cách Hamming thực hiện phép XOR từng bit và cộng kết quả để đếm số lượng bit khác nhau. Kết quả là 8 giá trị khoảng cách nhánh (Branch Metrics) được tính toán song song, đại diện cho chi phí để đi từ trạng thái hiện tại đến trạng thái kế tiếp.

Bảng 2.6: Giao diện khối BMU

Tên tín hiệu	Độ rộng	Hướng	Mô tả
piso_data_i	2	I	Dữ liệu 2-bit từ PISO.
bm_s0_s0_o	2	O	Chi phí nhánh từ S0 đến S0.
bm_s0_s2_o	2	O	Chi phí nhánh từ S0 đến S2.
bm_s1_s0_o	2	O	Chi phí nhánh từ S1 đến S0.
bm_s1_s2_o	2	O	Chi phí nhánh từ S1 đến S2.
bm_s2_s1_o	2	O	Chi phí nhánh từ S2 đến S1.
bm_s2_s3_o	2	O	Chi phí nhánh từ S2 đến S3.
bm_s3_s1_o	2	O	Chi phí nhánh từ S3 đến S1.
bm_s3_s3_o	2	O	Chi phí nhánh từ S3 đến S3.

2.2.4 Khối Cộng - So sánh - Chọn (ACSU)

Cập nhật Path Metric cho 4 trạng thái và đưa ra quyết định chọn đường dẫn (Survivor Path).



Hình 2.5: Sơ đồ khối ACSU

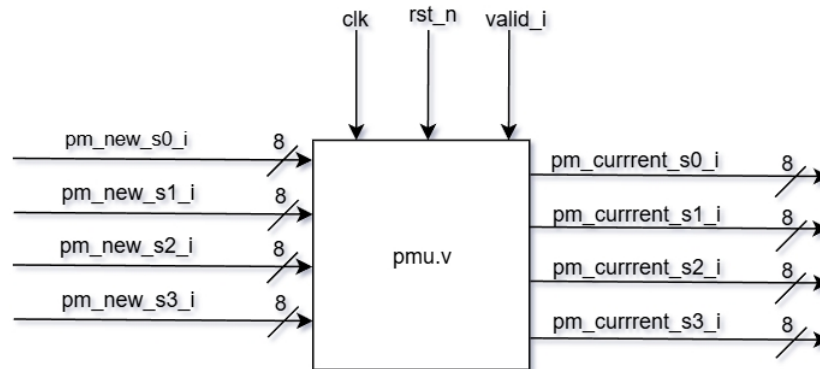
Mô tả luồng hoạt động: Khối ACSU nhận 8 Metric nhánh từ BMU và 4 Metric đường dẫn từ PMU. Với mỗi trạng thái đích, ACSU tính toán hai tổng chi phí tiềm năng (Candidate Path Metrics) từ hai trạng thái nguồn. Bộ so sánh xác định đường dẫn có chi phí nhỏ hơn và cập nhật giá trị đó làm Path Metric mới (pm_new_sX). Đồng thời, bit quyết định (0 hoặc 1) tương ứng với nhánh được chọn sẽ được gửi tới khối TBU để lưu lại lịch sử.

Bảng 2.7: Giao diện khối ACSU

Tên tín hiệu	Độ rộng	Hướng	Mô tả
bm_s0_s0_i	2	I	Chi phí nhánh từ S0 đến S0.
bm_s0_s2_i	2	I	Chi phí nhánh từ S0 đến S2.
bm_s1_s0_i	2	I	Chi phí nhánh từ S1 đến S0.
bm_s1_s2_i	2	I	Chi phí nhánh từ S1 đến S2.
bm_s2_s1_i	2	I	Chi phí nhánh từ S2 đến S1.
bm_s2_s3_i	2	I	Chi phí nhánh từ S2 đến S3.
bm_s3_s1_i	2	I	Chi phí nhánh từ S3 đến S1.
bm_s3_s3_i	2	I	Chi phí nhánh từ S3 đến S3.
pm_s0_i	8	I	Chi phí đường đi hiện tại của S0.
pm_s1_i	8	I	Chi phí đường đi hiện tại của S1.
pm_s2_i	8	I	Chi phí đường đi hiện tại của S2.
pm_s3_i	8	I	Chi phí đường đi hiện tại của S3.
pm_s0_o	8	O	Chi phí đường đi mới của S0.
pm_s1_o	8	O	Chi phí đường đi mới của S1.
pm_s2_o	8	O	Chi phí đường đi mới của S2.
pm_s3_o	8	O	Chi phí đường đi mới của S3.
dec_bits_o	4	O	4 bit quyết định chọn nhánh (1 bit mỗi trạng thái).

2.2.5 Khối quản lý Path Metric (PMU)

Lưu trữ và phản hồi chi phí tích lũy của các trạng thái.



Hình 2.6: Sơ đồ khối PMU

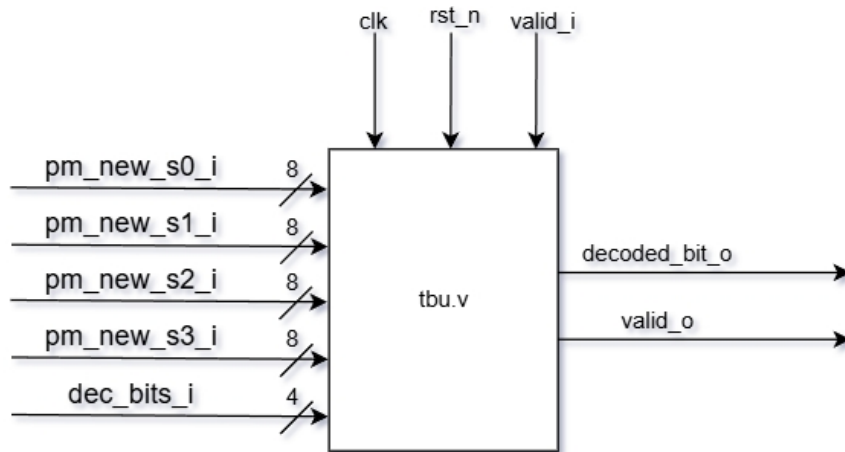
Mô tả luồng hoạt động: PMU đóng vai trò là bộ nhớ trạng thái của thuật toán Viterbi. Khi có tín hiệu kích hoạt (*valid_i*), PMU cập nhật 4 thanh ghi lưu trữ giá trị Metric mới nhất nhận được từ ACSU. Khi Reset, trạng thái S_0 được khởi tạo giá trị 0, trong khi các trạng thái khác được gán giá trị tối đa (255) để đảm bảo thuật toán luôn bắt đầu tìm kiếm từ trạng thái S_0 .

Bảng 2.8: Giao diện khối PMU

Tên tín hiệu	Độ rộng	Hướng	Mô tả
clk	1	I	Xung clock hệ thống.
rst_n	1	I	Reset hệ thống (Tích cực thấp).
valid_i	1	I	Cho phép cập nhật giá trị mới.
pm_new_s0_i	8	I	Giá trị mới từ ACSU cho S0.
pm_new_s1_i	8	I	Giá trị mới từ ACSU cho S1.
pm_new_s2_i	8	I	Giá trị mới từ ACSU cho S2.
pm_new_s3_i	8	I	Giá trị mới từ ACSU cho S3.
pm_current_s0_o	8	O	Giá trị hiện tại của S0 gửi ACSU.
pm_current_s1_o	8	O	Giá trị hiện tại của S1 gửi ACSU.
pm_current_s2_o	8	O	Giá trị hiện tại của S2 gửi ACSU.
pm_current_s3_o	8	O	Giá trị hiện tại của S3 gửi ACSU.

2.2.6 Khối truy vết Survivor Memory (TBU - Register Exchange)

Thực hiện kỹ thuật Register Exchange để khôi phục bit gốc từ các bit quyết định.



Hình 2.7: Sơ đồ khối TBU

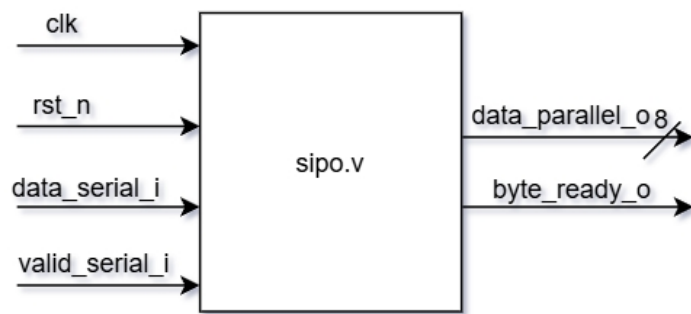
Mô tả luồng hoạt động: TBU quản lý 4 thanh ghi lịch sử 15-bit cho 4 trạng thái. Tại mỗi chu kỳ clock, dựa trên 4 bit quyết định từ ACSU, nội dung của các thanh ghi này được cập nhật theo quy tắc Register Exchange: Thanh ghi của trạng thái S_k sẽ nhận giá trị từ thanh ghi của trạng thái nguồn đã đi tới nó, cộng thêm bit quyết định mới. Sau khi pipeline đầy, bộ so sánh so sánh 4 giá trị Path Metric hiện tại để tìm trạng thái tối ưu nhất, và bit cũ nhất (MSB) trong thanh ghi lịch sử của trạng thái đó sẽ được chọn làm bit đầu ra (`decoded_bit_o`).

Bảng 2.9: Giao diện khối TBU

Tên tín hiệu	Độ rộng	Hướng	Mô tả
<code>clk</code>	1	I	Xung clock hệ thống.
<code>rst_n</code>	1	I	Reset hệ thống (Tích cực thấp).
<code>valid_i</code>	1	I	Tín hiệu cho phép cập nhật.
<code>dec_bits_i</code>	4	I	Các bit quyết định từ ACSU.
<code>pm_new_s0_i</code>	8	I	Chỉ phí đường dẫn mới của S0 (để so sánh).
<code>pm_new_s1_i</code>	8	I	Chỉ phí đường dẫn mới của S1 (để so sánh).
<code>pm_new_s2_i</code>	8	I	Chỉ phí đường dẫn mới của S2 (để so sánh).
<code>pm_new_s3_i</code>	8	I	Chỉ phí đường dẫn mới của S3 (để so sánh).
<code>decoded_bit_o</code>	1	O	Bit dữ liệu đã giải mã thành công.
<code>valid_o</code>	1	O	Báo bit giải mã hợp lệ.

2.2.7 Khối chuyển đổi nối tiếp sang song song (SIPO)

Gom các bit đơn lẻ từ TBU thành byte 8-bit hoàn chỉnh trước khi xuất ra ngoài hệ thống.



Hình 2.8: Sơ đồ khối SIPO

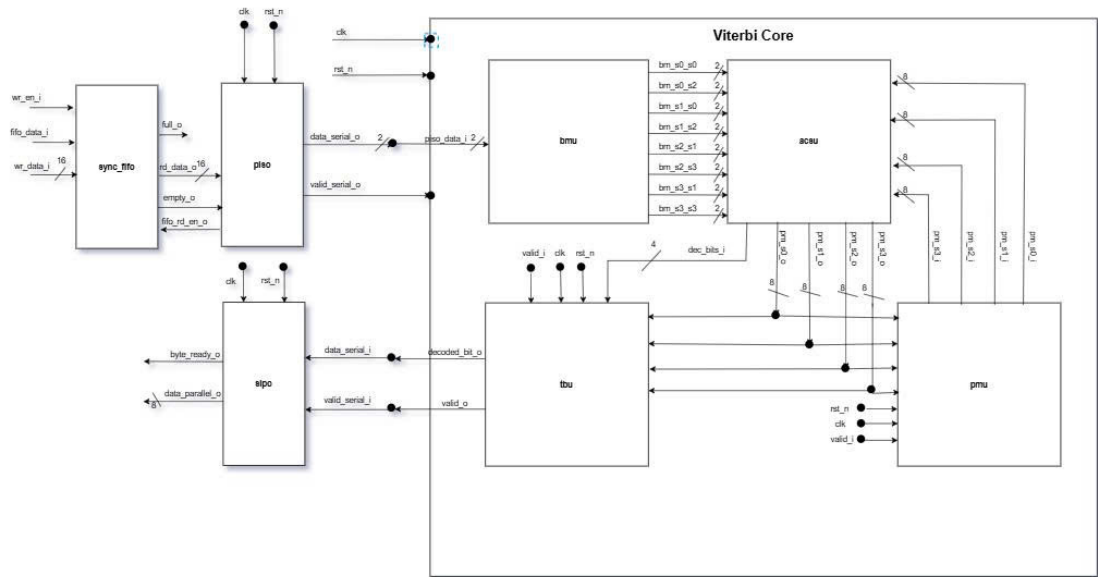
Mô tả luồng hoạt động: Khối SIPO gom các bit đơn lẻ được giải mã từ TBU thành byte dữ liệu hoàn chỉnh. Mỗi khi nhận một bit hợp lệ (`valid_serial_i`), bit này được dịch vào thanh ghi đếm từ vị trí MSB. Bộ đếm theo dõi số lượng bit đã nhận; khi đếm đủ 8 bit, tín hiệu `byte_ready_o` được kích hoạt, báo hiệu một byte dữ liệu (`data_parallel_o`) đã sẵn sàng để xuất ra output.

Bảng 2.10: Giao diện khối SIPO

Tên tín hiệu	Độ rộng	Hướng	Mô tả
<code>clk</code>	1	I	Xung clock hệ thống.
<code>rst_n</code>	1	I	Reset hệ thống (Tích cực thấp).
<code>data_serial_i</code>	1	I	Bit nhận từ TBU.
<code>valid_serial_i</code>	1	I	Cờ báo bit đầu vào hợp lệ.
<code>data_parallel_o</code>	8	O	Dữ liệu byte hoàn chỉnh.
<code>byte_ready_o</code>	1	O	Báo hiệu đã đủ 8 bit (Byte Valid).

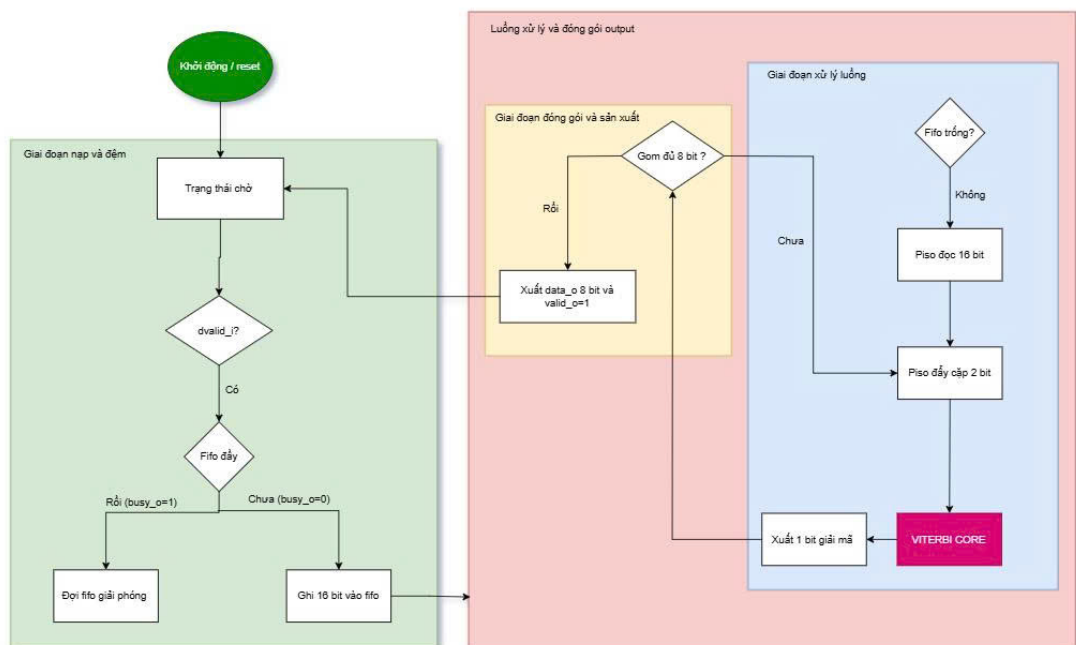
2.3 Triển khai RTL (RTL Implementation)

Mã nguồn RTL được viết bằng Verilog HDL. Trước khi đi vào chi tiết, sơ đồ khối hệ thống (Hình 2.9) và lưu đồ thuật toán (Hình 2.10) minh họa luồng xử lý tổng quan.



Hình 2.9: Sơ đồ khối chi tiết hệ thống Viterbi Decoder SoC

Lưu đồ thuật toán điều khiển luồng dữ liệu:



Hình 2.10: Lưu đồ thuật toán điều khiển luồng dữ liệu hệ thống

Thay vì sử dụng bộ nhớ RAM để lưu vết và quay lui (Traceback) gây độ trễ lớn, nhóm sử dụng kỹ thuật **Register Exchange (RE)** trong khối TBU.

Mỗi trạng thái (S_0, S_1, S_2, S_3) quản lý một thanh ghi dịch lịch sử dài 15 bit.

```
1 // Update logic example for state S0
2 if (dec_bits_i[0] == 0)
3     history_s0 <= {history_s0[TBL-2:0], 1'b0}; // Select path from
        previous S0
4 else
5     history_s0 <= {history_s1[TBL-2:0], 1'b0}; // Select path from
        previous S1
```

Listing 2.1: Cơ chế Register Exchange trong TBU

Nhờ cơ chế này, sau khi pipeline được điền đầy (15 chu kỳ), đầu ra dữ liệu là liên tục (streaming) với năng suất (throughput) cao nhất (1 bit/cycle).

Chương 3: KIỂM TRA CHỨC NĂNG MẠCH

3.1 Môi trường kiểm chứng (Testbench Environment)

Môi trường kiểm tra được xây dựng trong file `tb_system_top.sv` với các thành phần:

- **Generator:** Tạo ngẫu nhiên 1025 gói dữ liệu 16-bit.
- **Driver:** Đưa dữ liệu vào hệ thống qua giao thức bắt tay (valid/busy).
- **Monitor:** Thu thập dữ liệu đầu ra `data_o`.
- **Scoreboard:** So sánh kết quả mô phỏng với "Golden Model"(mô hình chuẩn).

3.2 Kết quả mô phỏng (Simulation Results)

Dưới đây là kết quả mô phỏng chi tiết cho từng khối chức năng (Unit Level) và toàn hệ thống (System Level). Các hình ảnh bao gồm log dữ liệu và giản đồ sóng xác nhận hoạt động chính xác của thiết kế.

3.2.1 Khối đệm dữ liệu (Sync FIFO)

Bảng dưới đây mô tả các kịch bản kiểm thử được thực hiện cho khối Sync FIFO:

ID	Tên Kịch Bản	Hành động	Kỳ vọng	Kết quả
TC_01	Reset & Init	Kích hoạt <code>rst_n</code> xuống 0 rồi lên 1.	<code>empty_o=1</code> , <code>full_o=0</code> .	PASS
TC_02	Write until Full	Ghi liên tục 16 phần tử.	<code>full_o</code> chuyển lên 1.	PASS
TC_03	Read until Empty	Đọc liên tục 16 phần tử.	<code>empty_o</code> chuyển lên 1, dữ liệu khớp.	PASS
TC_04	Data Integrity	Lặp Ghi đầy - Đọc rỗng 5 vòng.	Dữ liệu khớp mỗi vòng.	PASS
TC_05	Timing Check	Kiểm tra độ trễ dữ liệu ra.	Dữ liệu ổn định sau cạnh clock.	PASS

Bảng 3.1: Các kịch bản kiểm thử khối Sync FIFO

```

iroot@LAPTOP-7Q2ILT09:/mnt/e/vlsi/vlsi_ver1
START FIFO TEST: AUTO-RESET INDEX VERSION

--- Round 1 ---
[210000 ns] FIFO FULL - Start Reading...
[237000 ns] [PASS] Got: 3524 | Exp: 3524
[247000 ns] [PASS] Got: 5e81 | Exp: 5e81
[257000 ns] [PASS] Got: d609 | Exp: d609
[267000 ns] [PASS] Got: 5663 | Exp: 5663
[277000 ns] [PASS] Got: 7b0d | Exp: 7b0d
[287000 ns] [PASS] Got: 998d | Exp: 998d
[297000 ns] [PASS] Got: 8465 | Exp: 8465
[307000 ns] [PASS] Got: 5212 | Exp: 5212
[317000 ns] [PASS] Got: e301 | Exp: e301
[327000 ns] [PASS] Got: c0d0 | Exp: c0d0
[337000 ns] [PASS] Got: f176 | Exp: f176
[347000 ns] [PASS] Got: cd3d | Exp: cd3d
[357000 ns] [PASS] Got: 57ed | Exp: 57ed
[367000 ns] [PASS] Got: f78c | Exp: f78c
[377000 ns] [PASS] Got: e9f9 | Exp: e9f9
[387000 ns] [PASS] Got: 24c6 | Exp: 24c6

--- Round 2 ---
[610000 ns] FIFO FULL - Start Reading...
[637000 ns] [PASS] Got: d2aa | Exp: d2aa
[647000 ns] [PASS] Got: f7e5 | Exp: f7e5
[657000 ns] [PASS] Got: 7277 | Exp: 7277
[667000 ns] [PASS] Got: d612 | Exp: d612
[677000 ns] [PASS] Got: db8f | Exp: db8f
[687000 ns] [PASS] Got: 69f2 | Exp: 69f2
[697000 ns] [PASS] Got: 96ce | Exp: 96ce
[707000 ns] [PASS] Got: 7ae8 | Exp: 7ae8
[717000 ns] [PASS] Got: 4ec5 | Exp: 4ec5
[727000 ns] [PASS] Got: 495c | Exp: 495c
[737000 ns] [PASS] Got: 28bd | Exp: 28bd
[747000 ns] [PASS] Got: 582d | Exp: 582d
[757000 ns] [PASS] Got: 2665 | Exp: 2665
[767000 ns] [PASS] Got: 6263 | Exp: 6263
[777000 ns] [PASS] Got: 870a | Exp: 870a
[787000 ns] [PASS] Got: 2280 | Exp: 2280

--- Round 3 ---
[1010000 ns] FIFO FULL - Start Reading...
[1037000 ns] [PASS] Got: 45aa | Exp: 45aa
[1047000 ns] [PASS] Got: cc9d | Exp: cc9d
[1057000 ns] [PASS] Got: 3e96 | Exp: 3e96
[1067000 ns] [PASS] Got: b813 | Exp: b813
[1077000 ns] [PASS] Got: 380d | Exp: 380d

```

Hình 3.1: Log mô phỏng Sync FIFO - Phần 1

```

iroot@LAPTOP-7Q2ILT09:/mnt/e/vlsi/vlsi_ver1
[1137000 ns] [PASS] Got: e91d | Exp: e91d
[1147000 ns] [PASS] Got: 72cf | Exp: 72cf
[1157000 ns] [PASS] Got: 4923 | Exp: 4923
[1167000 ns] [PASS] Got: 650a | Exp: 650a
[1177000 ns] [PASS] Got: 0aca | Exp: 0aca
[1187000 ns] [PASS] Got: 4c3c | Exp: 4c3c

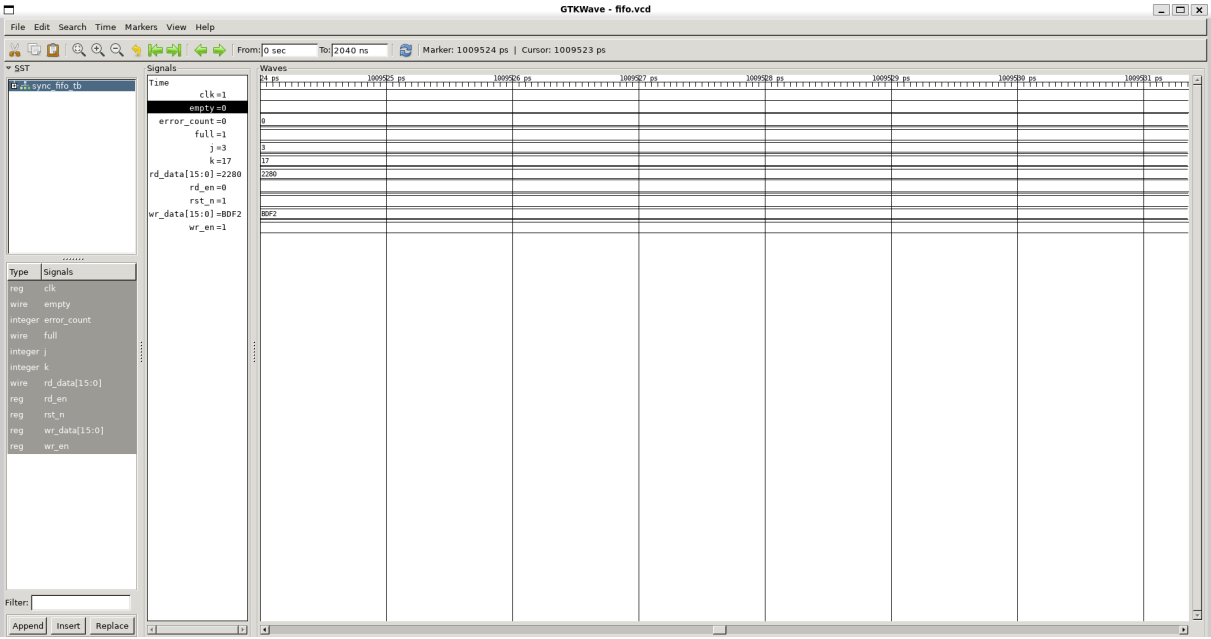
--- Round 4 ---
[1410000 ns] FIFO FULL - Start Reading...
[1437000 ns] [PASS] Got: 618a | Exp: 618a
[1447000 ns] [PASS] Got: b341 | Exp: b341
[1457000 ns] [PASS] Got: 34d8 | Exp: 34d8
[1467000 ns] [PASS] Got: f378 | Exp: f378
[1477000 ns] [PASS] Got: 1289 | Exp: 1289
[1487000 ns] [PASS] Got: 0deb | Exp: 0deb
[1497000 ns] [PASS] Got: 65b6 | Exp: 65b6
[1507000 ns] [PASS] Got: f9c6 | Exp: f9c6
[1517000 ns] [PASS] Got: 13ae | Exp: 13ae
[1527000 ns] [PASS] Got: 02bc | Exp: 02bc
[1537000 ns] [PASS] Got: dd2a | Exp: dd2a
[1547000 ns] [PASS] Got: 9a0b | Exp: 9a0b
[1557000 ns] [PASS] Got: be71 | Exp: be71
[1567000 ns] [PASS] Got: 4185 | Exp: 4185
[1577000 ns] [PASS] Got: 554f | Exp: 554f
[1587000 ns] [PASS] Got: 603b | Exp: 603b

--- Round 5 ---
[1810000 ns] FIFO FULL - Start Reading...
[1837000 ns] [PASS] Got: 327e | Exp: 327e
[1847000 ns] [PASS] Got: 4b15 | Exp: 4b15
[1857000 ns] [PASS] Got: 9bf1 | Exp: 9bf1
[1867000 ns] [PASS] Got: 4bd9 | Exp: 4bd9
[1877000 ns] [PASS] Got: 0762 | Exp: 0762
[1887000 ns] [PASS] Got: fb4c | Exp: fb4c
[1897000 ns] [PASS] Got: 559f | Exp: 559f
[1907000 ns] [PASS] Got: a18f | Exp: a18f
[1917000 ns] [PASS] Got: a9f8 | Exp: a9f8
[1927000 ns] [PASS] Got: 60b7 | Exp: 60b7
[1937000 ns] [PASS] Got: 569f | Exp: 569f
[1947000 ns] [PASS] Got: 945c | Exp: 945c
[1957000 ns] [PASS] Got: c95b | Exp: c95b
[1967000 ns] [PASS] Got: 3789 | Exp: 3789
[1977000 ns] [PASS] Got: 3249 | Exp: 3249
[1987000 ns] [PASS] Got: 3ed0 | Exp: 3ed0

>>> TONG KET: ALL PASS (0 ERRORS) <<<
iroot@LAPTOP-7Q2ILT09:/mnt/e/vlsi/vlsi_ver1$

```

Hình 3.2: Log mô phỏng Sync FIFO - Phần 2



Hình 3.3: Giải đồ sóng tín hiệu Sync FIFO

3.2.2 Khối chuyển đổi song song sang nối tiếp (PISO)

Bảng dưới đây mô tả các kịch bản kiểm thử được thực hiện cho khối PISO:

ID	Tên Kịch Bản	Dữ liệu vào	Kỳ vọng	Kết quả
TC_01	Alternating (0xAAAA)	16'hAAAA	Xuất 8 cặp: 10,10,10...	PASS
TC_02	Inverse (0x5555)	16'h5555	Xuất 8 cặp: 01,01,01...	PASS
TC_03	All Ones (0xFFFF)	16'hFFFF	Xuất 8 cặp: 11,11,11...	PASS
TC_04	All Zeros (0x0000)	16'h0000	Xuất 8 cặp: 00,00,00...	PASS
TC_05	Random Stress	Ngẫu nhiên	Dữ liệu khớp sau 5 vòng.	PASS

Bảng 3.2: Các kịch bản kiểm thử khối PISO

```

Iroot@LAPTOP-7QZILT09:/mnt/e/vlsi/vlsi_ver1
[306000 ns] [PASS] Step 2 | Got: 10 | Exp: 10
[310000 ns] [PASS] Step 3 | Got: 10 | Exp: 10
[320000 ns] [PASS] Step 4 | Got: 10 | Exp: 10
[330000 ns] [PASS] Step 5 | Got: 10 | Exp: 10
[340000 ns] [PASS] Step 6 | Got: 10 | Exp: 10
[350000 ns] [PASS] Step 7 | Got: 10 | Exp: 10
Iteration 4:
[406000 ns] [PASS] Step 0 | Got: 10 | Exp: 10
[410000 ns] [PASS] Step 1 | Got: 10 | Exp: 10
[420000 ns] [PASS] Step 2 | Got: 10 | Exp: 10
[430000 ns] [PASS] Step 3 | Got: 10 | Exp: 10
[440000 ns] [PASS] Step 4 | Got: 10 | Exp: 10
[450000 ns] [PASS] Step 5 | Got: 10 | Exp: 10
[460000 ns] [PASS] Step 6 | Got: 10 | Exp: 10
[470000 ns] [PASS] Step 7 | Got: 10 | Exp: 10
Iteration 5:
[520000 ns] [PASS] Step 0 | Got: 10 | Exp: 10
[530000 ns] [PASS] Step 1 | Got: 10 | Exp: 10
[540000 ns] [PASS] Step 2 | Got: 10 | Exp: 10
[550000 ns] [PASS] Step 3 | Got: 10 | Exp: 10
[560000 ns] [PASS] Step 4 | Got: 10 | Exp: 10
[570000 ns] [PASS] Step 5 | Got: 10 | Exp: 10
[580000 ns] [PASS] Step 6 | Got: 10 | Exp: 10
[590000 ns] [PASS] Step 7 | Got: 10 | Exp: 10
---> SCENARIO 2: Data 16'h5555
Iteration 1:
[640000 ns] [PASS] Step 0 | Got: 01 | Exp: 01
[650000 ns] [PASS] Step 1 | Got: 01 | Exp: 01
[660000 ns] [PASS] Step 2 | Got: 01 | Exp: 01
[670000 ns] [PASS] Step 3 | Got: 01 | Exp: 01
[680000 ns] [PASS] Step 4 | Got: 01 | Exp: 01
[690000 ns] [PASS] Step 5 | Got: 01 | Exp: 01
[700000 ns] [PASS] Step 6 | Got: 01 | Exp: 01
[710000 ns] [PASS] Step 7 | Got: 01 | Exp: 01
Iteration 2:
[760000 ns] [PASS] Step 0 | Got: 01 | Exp: 01
[770000 ns] [PASS] Step 1 | Got: 01 | Exp: 01
[780000 ns] [PASS] Step 2 | Got: 01 | Exp: 01
[790000 ns] [PASS] Step 3 | Got: 01 | Exp: 01
[800000 ns] [PASS] Step 4 | Got: 01 | Exp: 01
[810000 ns] [PASS] Step 5 | Got: 01 | Exp: 01
[820000 ns] [PASS] Step 6 | Got: 01 | Exp: 01
[830000 ns] [PASS] Step 7 | Got: 01 | Exp: 01
Iteration 3:
[880000 ns] [PASS] Step 0 | Got: 01 | Exp: 01
[890000 ns] [PASS] Step 1 | Got: 01 | Exp: 01
[900000 ns] [PASS] Step 2 | Got: 01 | Exp: 01
[910000 ns] [PASS] Step 3 | Got: 01 | Exp: 01

```

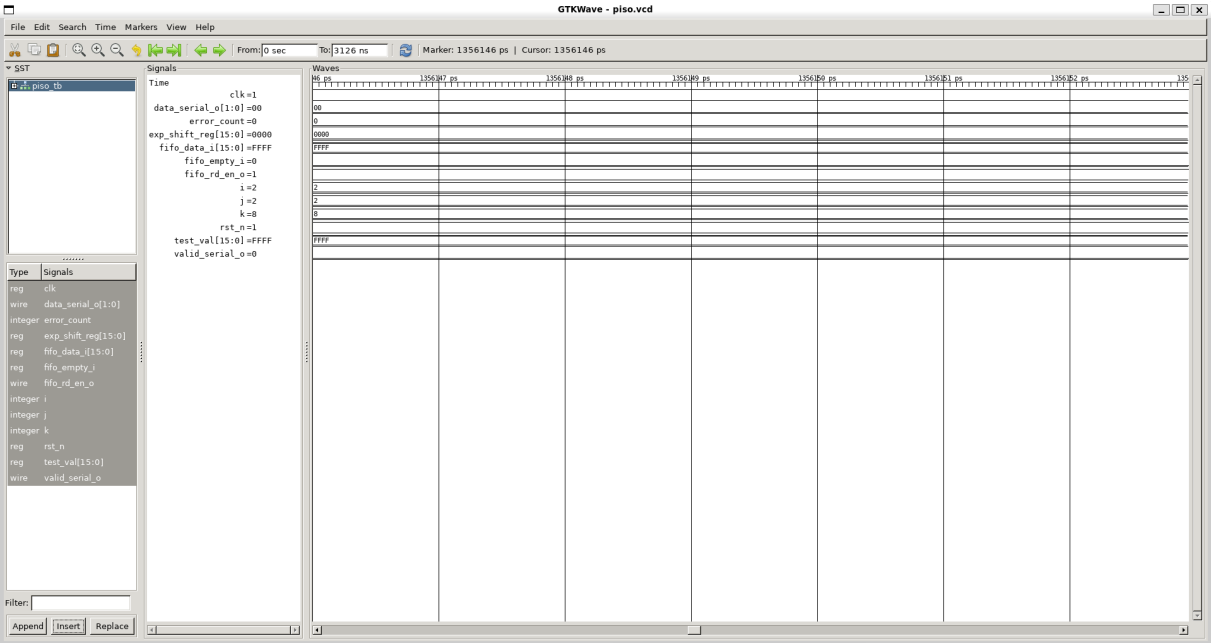
Hình 3.4: Log mô phỏng PISO - Phần 1

```

Iroot@LAPTOP-7QZILT09:/mnt/e/vlsi/vlsi_ver1
[2446000 ns] [PASS] Step 0 | Got: 00 | Exp: 00
[2456000 ns] [PASS] Step 1 | Got: 11 | Exp: 11
[2466000 ns] [PASS] Step 2 | Got: 01 | Exp: 01
[2476000 ns] [PASS] Step 3 | Got: 01 | Exp: 01
[2486000 ns] [PASS] Step 4 | Got: 00 | Exp: 00
[2496000 ns] [PASS] Step 5 | Got: 10 | Exp: 10
[2506000 ns] [PASS] Step 6 | Got: 01 | Exp: 01
[2516000 ns] [PASS] Step 7 | Got: 00 | Exp: 00
Iteration 2:
[2566000 ns] [PASS] Step 0 | Got: 00 | Exp: 00
[2576000 ns] [PASS] Step 1 | Got: 11 | Exp: 11
[2586000 ns] [PASS] Step 2 | Got: 01 | Exp: 01
[2596000 ns] [PASS] Step 3 | Got: 01 | Exp: 01
[2606000 ns] [PASS] Step 4 | Got: 00 | Exp: 00
[2616000 ns] [PASS] Step 5 | Got: 10 | Exp: 10
[2626000 ns] [PASS] Step 6 | Got: 01 | Exp: 01
[2636000 ns] [PASS] Step 7 | Got: 00 | Exp: 00
Iteration 3:
[2686000 ns] [PASS] Step 0 | Got: 00 | Exp: 00
[2696000 ns] [PASS] Step 1 | Got: 11 | Exp: 11
[2706000 ns] [PASS] Step 2 | Got: 01 | Exp: 01
[2716000 ns] [PASS] Step 3 | Got: 01 | Exp: 01
[2726000 ns] [PASS] Step 4 | Got: 00 | Exp: 00
[2736000 ns] [PASS] Step 5 | Got: 10 | Exp: 10
[2746000 ns] [PASS] Step 6 | Got: 01 | Exp: 01
[2756000 ns] [PASS] Step 7 | Got: 00 | Exp: 00
Iteration 4:
[2806000 ns] [PASS] Step 0 | Got: 00 | Exp: 00
[2816000 ns] [PASS] Step 1 | Got: 11 | Exp: 11
[2826000 ns] [PASS] Step 2 | Got: 01 | Exp: 01
[2836000 ns] [PASS] Step 3 | Got: 01 | Exp: 01
[2846000 ns] [PASS] Step 4 | Got: 00 | Exp: 00
[2856000 ns] [PASS] Step 5 | Got: 10 | Exp: 10
[2866000 ns] [PASS] Step 6 | Got: 01 | Exp: 01
[2876000 ns] [PASS] Step 7 | Got: 00 | Exp: 00
Iteration 5:
[2926000 ns] [PASS] Step 0 | Got: 00 | Exp: 00
[2936000 ns] [PASS] Step 1 | Got: 11 | Exp: 11
[2946000 ns] [PASS] Step 2 | Got: 01 | Exp: 01
[2956000 ns] [PASS] Step 3 | Got: 01 | Exp: 01
[2966000 ns] [PASS] Step 4 | Got: 00 | Exp: 00
[2976000 ns] [PASS] Step 5 | Got: 10 | Exp: 10
[2986000 ns] [PASS] Step 6 | Got: 01 | Exp: 01
[2996000 ns] [PASS] Step 7 | Got: 00 | Exp: 00
=====
FINAL TOTAL ERRORS: 0 -> STATUS: ALL PASS
=====
Iroot@LAPTOP-7QZILT09:/mnt/e/vlsi/vlsi_ver1$

```

Hình 3.5: Log mô phỏng PISO - Phần 2



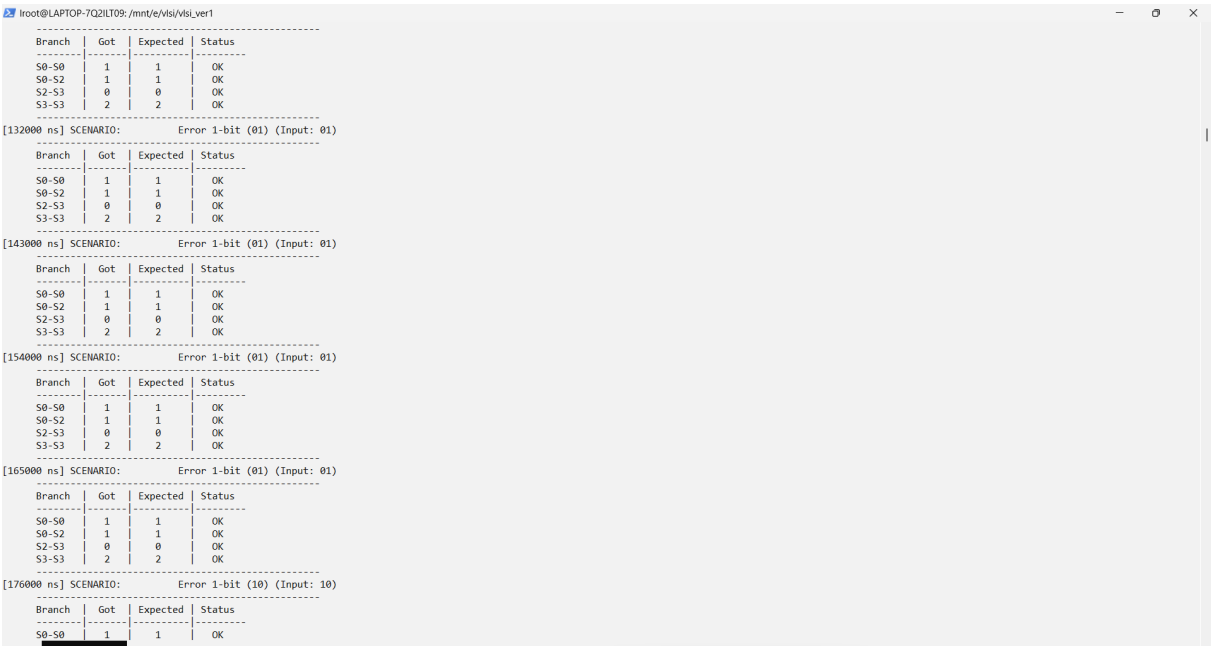
Hình 3.6: Giảm đồ sóng tín hiệu PISO

3.2.3 Khối tính toán nhánh (BMU)

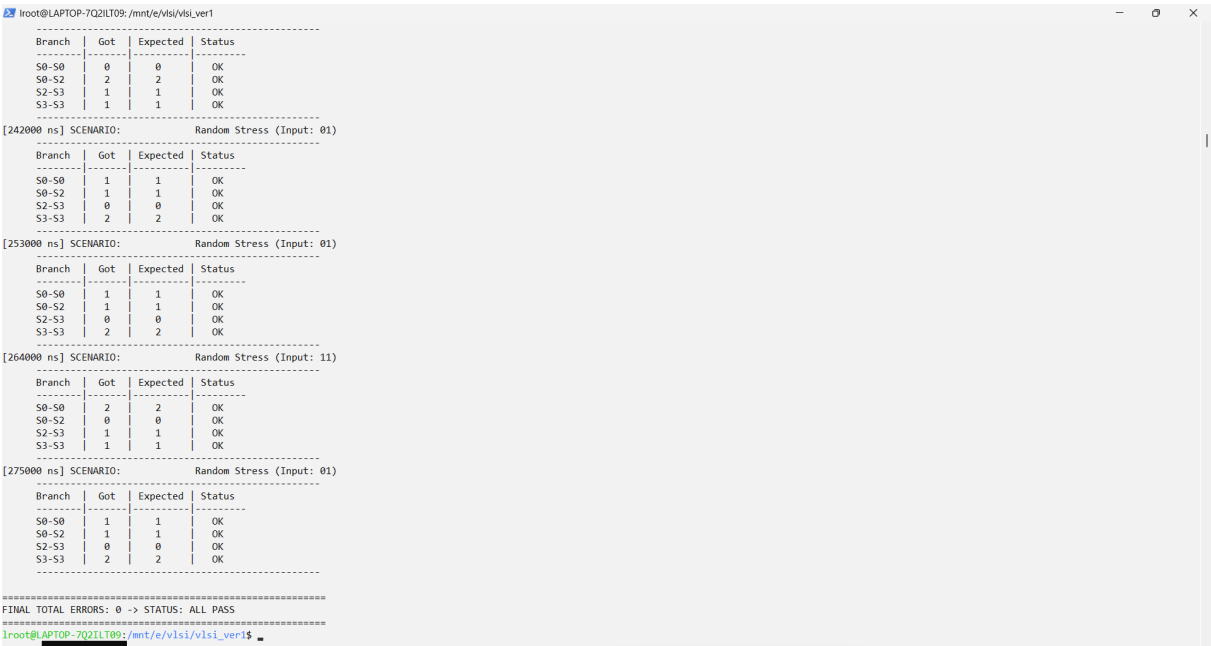
Bảng dưới đây mô tả các kịch bản kiểm thử được thực hiện cho khối BMU:

ID	Tên Kịch Bản	Input	Kỳ vọng (Branch Metrics)	Kết quả
TC_01	Ideal Case (00)	2'b00	S0-S0:0, S1-S2:0, S0-S2:2, S1-S0:2	PASS
TC_02	Inverse Case (11)	2'b11	S0-S0:2, S1-S2:2, S0-S2:0, S1-S0:0	PASS
TC_03	Single Bit (01)	2'b01	Nhánh kỳ vọng 01/10: 0 hoặc 1	PASS
TC_04	Single Bit (10)	2'b10	Tương tự TC_03, đảo vị trí	PASS
TC_05	Random Stress	Ngẫu nhiên	Ngõ ra thay đổi tức thời	PASS

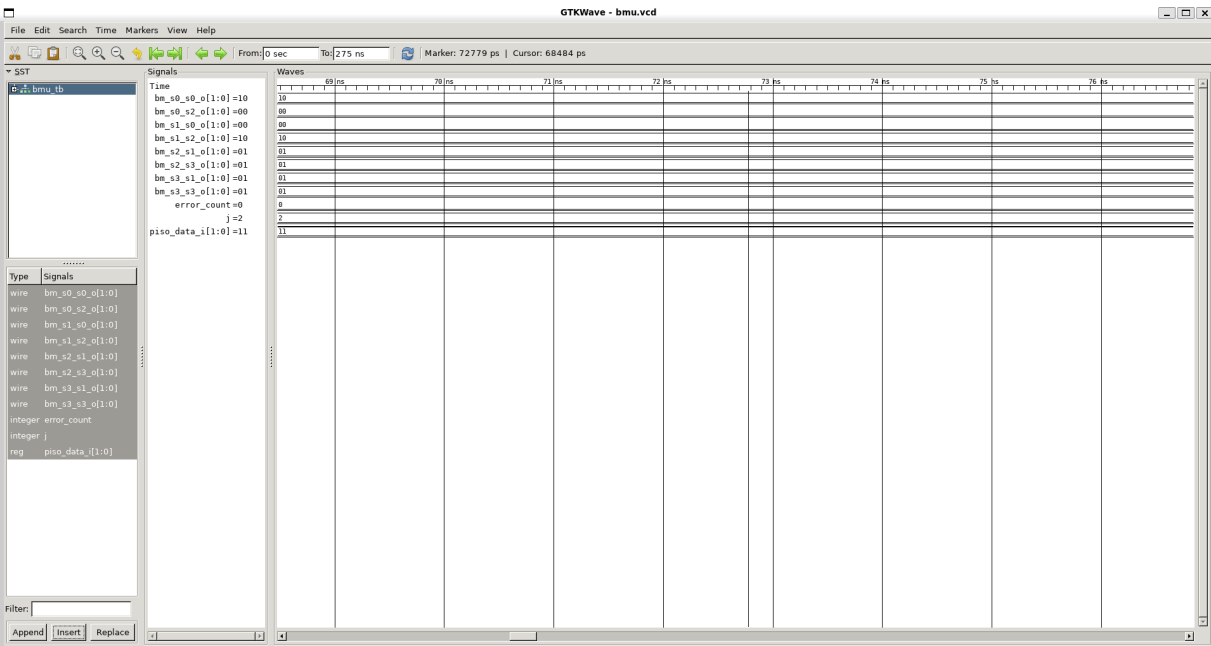
Bảng 3.3: Các kịch bản kiểm thử khối BMU



Hình 3.7: Log mô phỏng BMU - Phần 1



Hình 3.8: Log mô phỏng BMU - Phần 2



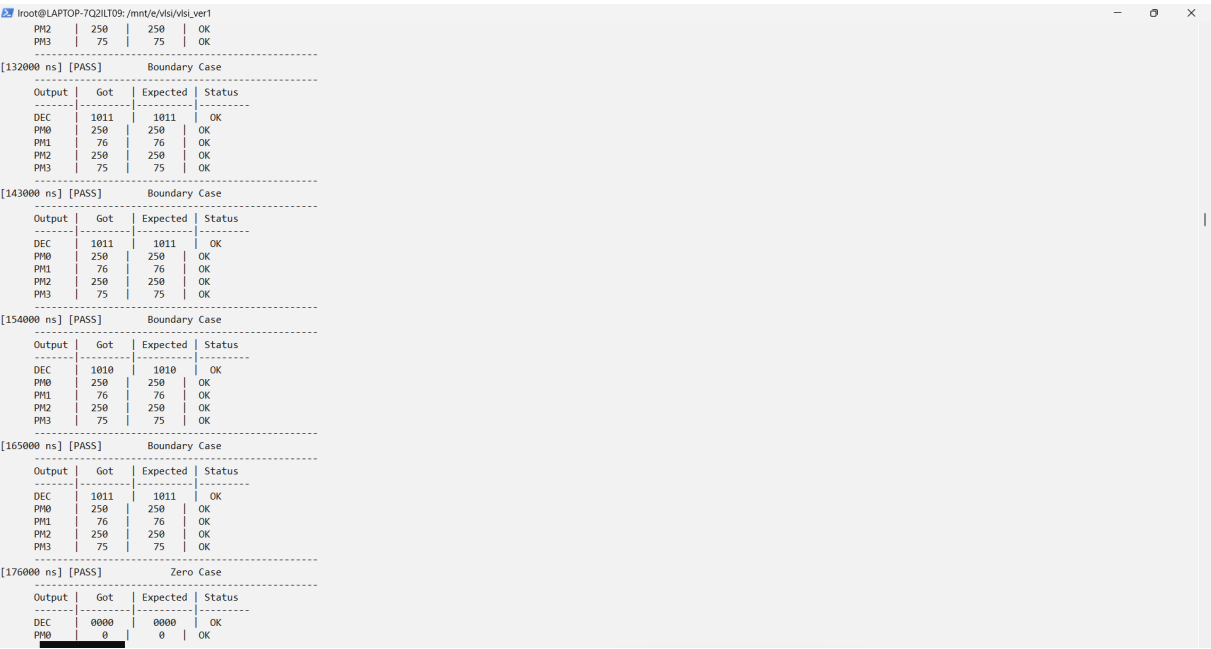
Hình 3.9: Giải đồ sóng tín hiệu BMU

3.2.4 Khối Cộng - So sánh - Chọn (ACSU)

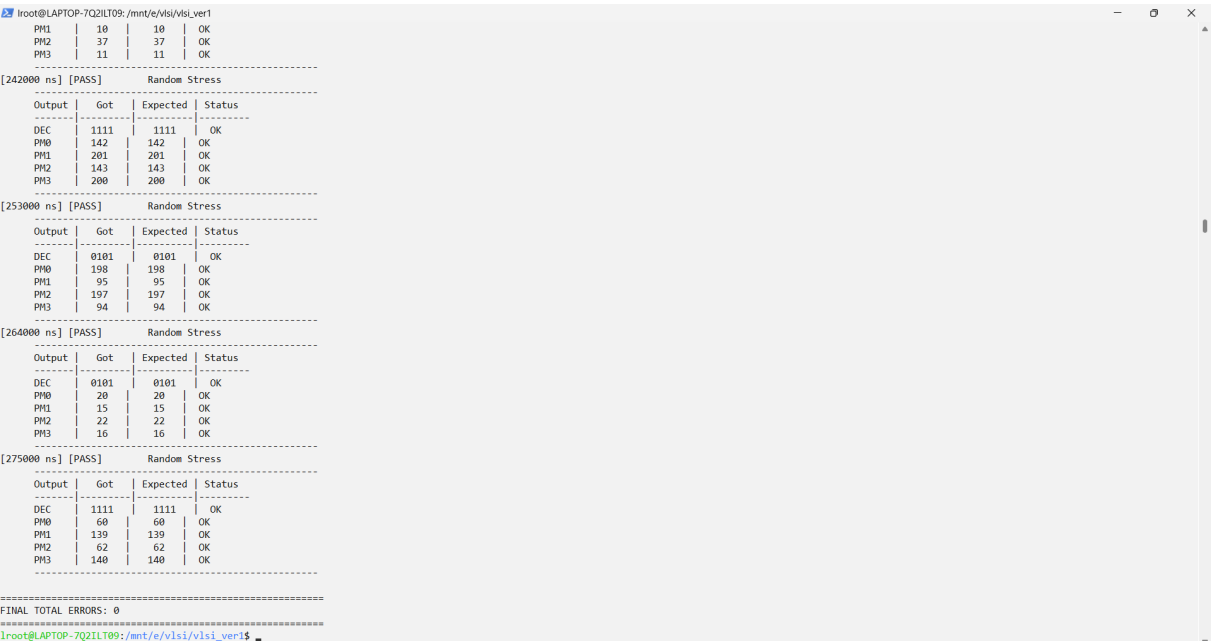
Bảng dưới đây mô tả các kịch bản kiểm thử được thực hiện cho khối ACSU:

ID	Tên Kịch Bản	Dữ liệu vào	Kỳ vọng	Kết quả
TC_01	Min Path Case	PM tăng dần, BM=1	Chọn nhánh PM+BM nhỏ nhất	PASS
TC_02	Switching Winner	Đảo PM0 và PM1	DEC bit thay đổi tương ứng	PASS
TC_03	Boundary Case	PM = 250	Phép cộng không tràn, so sánh đúng	PASS
TC_04	Zero Case	PM=0, BM=0	PM out=0, DEC chọn nhánh 0	PASS
TC_05	Random Stress	Ngẫu nhiên	$PM_{out} = \min(PMa+BMa, PMb+BMb)$	PASS

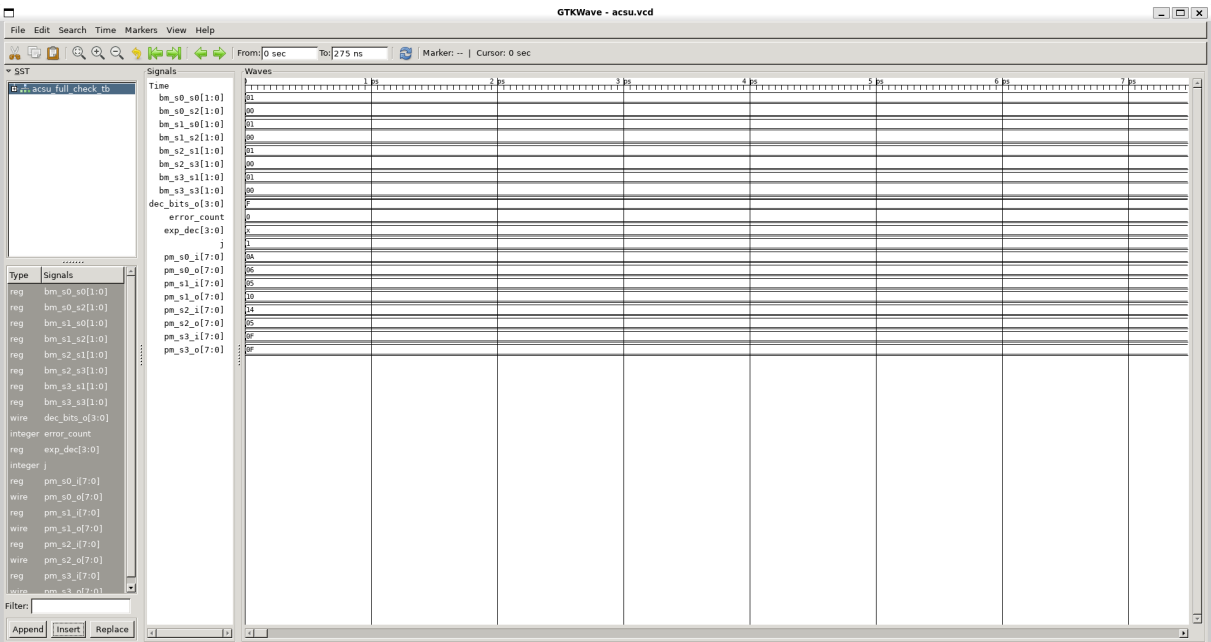
Bảng 3.4: Các kịch bản kiểm thử khối ACSU



Hình 3.10: Log mô phỏng ACSU - Phần 1



Hình 3.11: Log mô phỏng ACSU - Phần 2



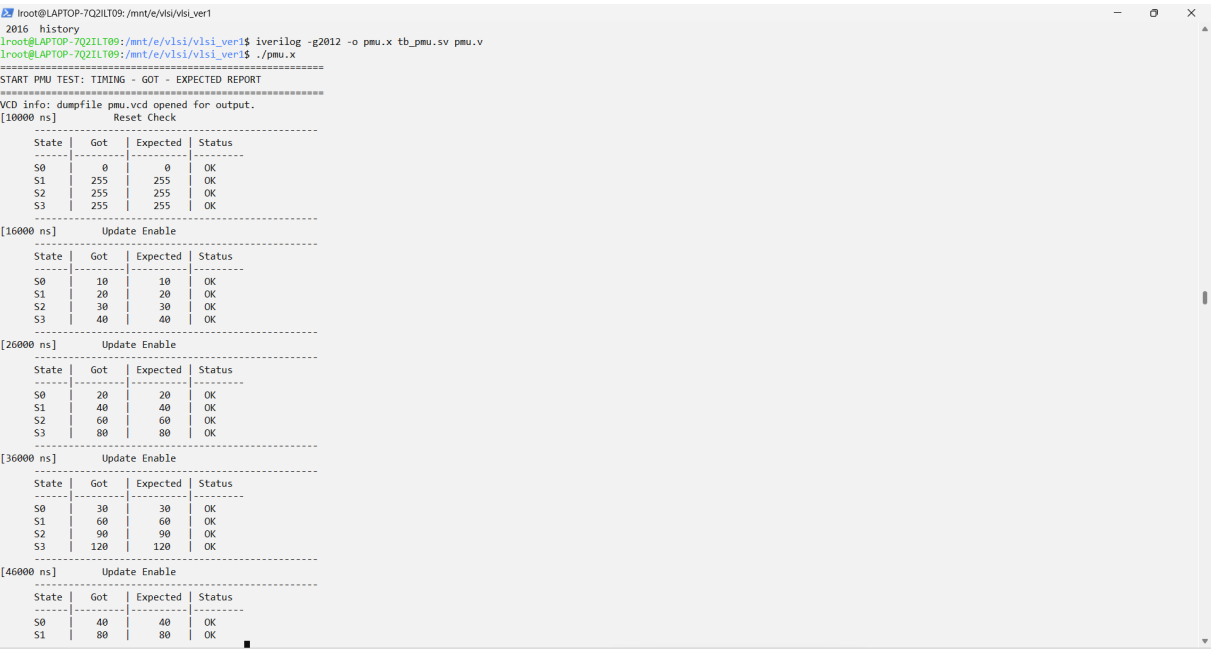
Hình 3.12: Giải đồ sóng tín hiệu ACSU

3.2.5 Khối quản lý Metric (PMU)

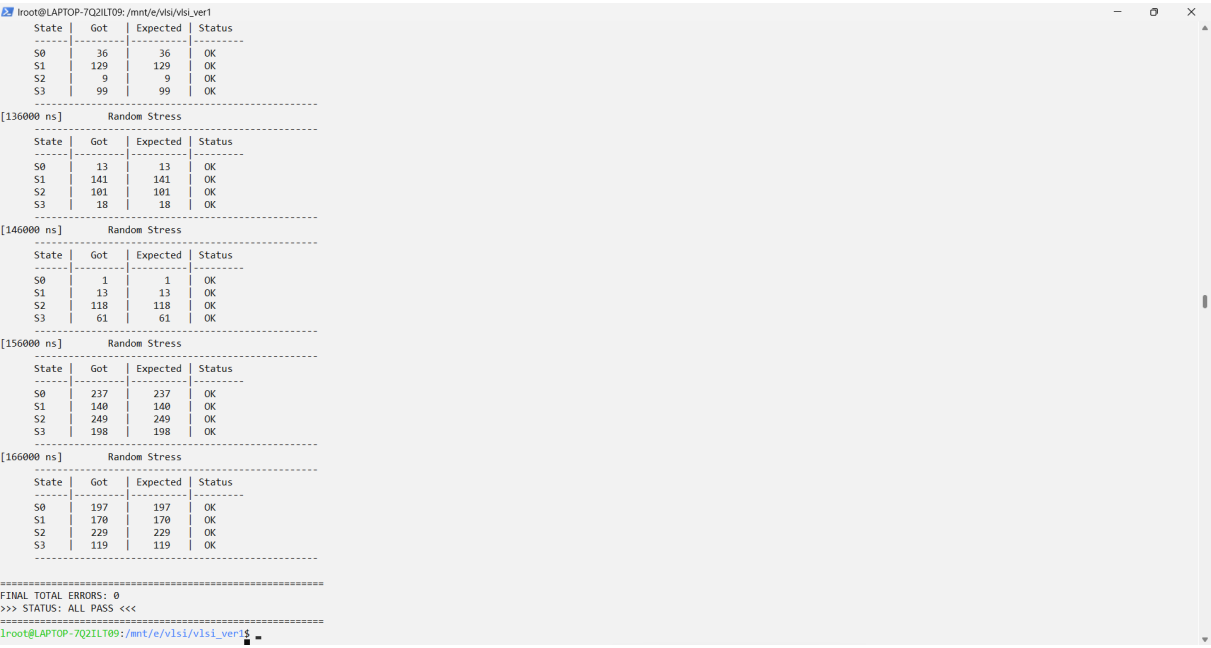
Bảng dưới đây mô tả các kịch bản kiểm thử được thực hiện cho khối PMU:

ID	Tên Kịch Bản	Hành động	Kỳ vọng	Kết quả
TC_01	Reset Check	Kéo rst_n xuống 0	S0=0, S1-S3=255	PASS
TC_02	Update Enable	valid_i=1, cấp dữ liệu mới	pm_current_o cập nhật đúng	PASS
TC_03	Keep Data	valid_i=0, thay đổi input	Ngõ ra giữ nguyên	PASS
TC_04	Zero Stream	Tất cả input = 0	Tất cả pm_current_o = 0	PASS
TC_05	Random Stress	Ngẫu nhiên 5 chu kỳ	Ngõ ra cập nhật chính xác	PASS

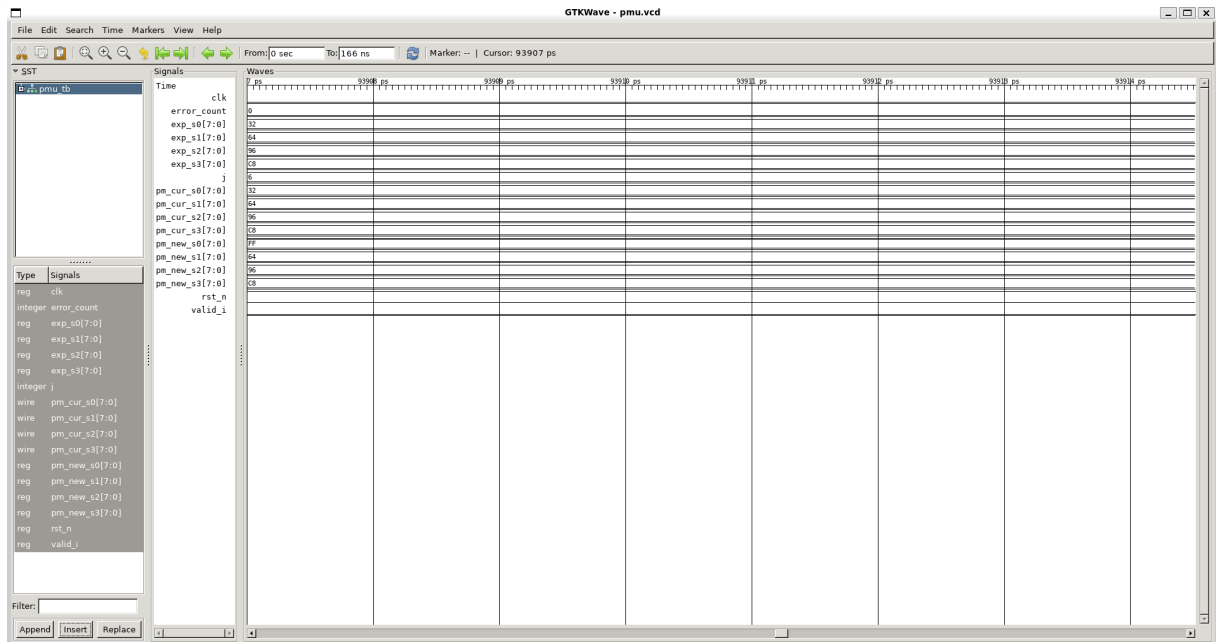
Bảng 3.5: Các kịch bản kiểm thử khối PMU



Hình 3.13: Log mô phỏng PMU - Phần 1



Hình 3.14: Log mô phỏng PMU - Phần 2



Hình 3.15: Giảm đồ sóng tín hiệu PMU

3.2.6 Khối truy vết (TBU)

Bảng dưới đây mô tả các kịch bản kiểm thử được thực hiện cho khối TBU:

ID	Tên Kịch Bản	Hành động	Kỳ vọng	Kết quả
TC_01	Reset Check	Kích hoạt rst_n	valid_o=0, de-coded_bit=0	PASS
TC_02	Pipeline Filling	Nạp 14 chu kỳ	valid_o giữ mức 0	PASS
TC_03	Start Decoding	Chu kỳ thứ 15	valid_o chuyển lên 1	PASS
TC_04	Data Traceback	Nạp chuỗi bit '1' qua S2	decoded_bit_o = 1	PASS
TC_05	Valid_i Gating	Tắt valid_i	valid_o về 0 ngay	PASS
TC_06	Winner Switching	PM_S3 nhỏ nhất	Chọn bit từ history_s3	PASS

Bảng 3.6: Các kịch bản kiểm thử khối TBU

```
iroot@LAPTOP-7QZILT09: /mnt/e/vlsi/vlsi_ver1
iroot@LAPTOP-7QZILT09: /mnt/e/vlsi/vlsi_ver1$ ./tbu.x
=====
BAT DAU TBU TEST: THOI GIAN - NHAN - KY VONG
=====
VCD info: dumpfile tbu.vcd opened for output.
[20000 ns] Reset Check
=====
Signal      Got      Expected  Status
-----
Valid_Out    0         0        OK
Decoded_Bit  0         0        OK
=====

---> Scenario 1: Filling Pipeline (Latency Test)
[166000 ns] line Processing Last Fill
=====
Signal      Got      Expected  Status
-----
Valid_Out    0         0        OK
Decoded_Bit  0         0        OK
=====

[176000 ns] peline Full - Start Valid
=====
Signal      Got      Expected  Status
-----
Valid_Out    1         1        OK
Decoded_Bit  0         0        OK
=====

---> Scenario 2: Data Traceback (S2 Input)
[336000 ns] Traceback Bit 1 Check
=====
Signal      Got      Expected  Status
-----
Valid_Out    1         1        OK
Decoded_Bit  1         1        OK
=====

---> Scenario 3: Valid_i Gating
[346000 ns] Valid_i Off
=====
Signal      Got      Expected  Status
-----
Valid_Out    0         0        OK
Decoded_Bit  1         0        OK
=====

---> Scenario 4: Winner Switching
[356000 ns] Winner switched to S3
=====
```

Hình 3.16: Log mô phỏng TBU - Phần 1

```
iroot@LAPTOP-7QZILT09: /mnt/e/vlsi/vlsi_ver1
---> Scenario 1: Filling Pipeline (Latency Test)
[166000 ns] line Processing Last Fill
=====
Signal      Got      Expected  Status
-----
Valid_Out    0         0        OK
Decoded_Bit  0         0        OK
=====

[176000 ns] peline Full - Start Valid
=====
Signal      Got      Expected  Status
-----
Valid_Out    1         1        OK
Decoded_Bit  0         0        OK
=====

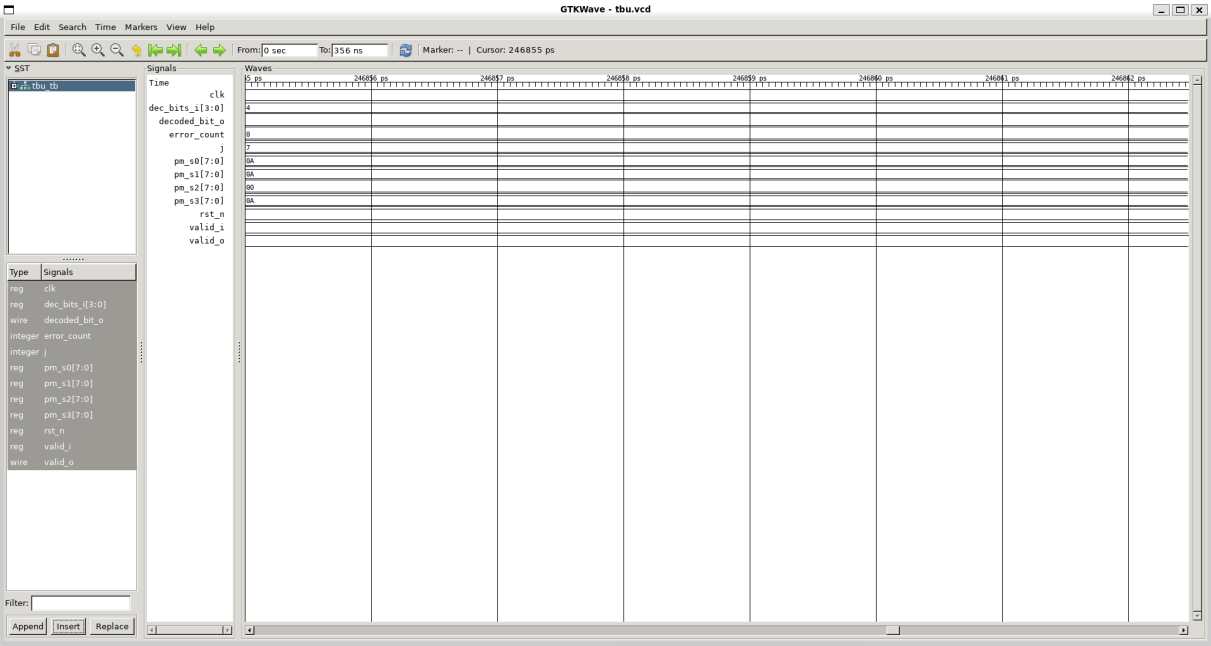
---> Scenario 2: Data Traceback (S2 Input)
[336000 ns] Traceback Bit 1 Check
=====
Signal      Got      Expected  Status
-----
Valid_Out    1         1        OK
Decoded_Bit  1         1        OK
=====

---> Scenario 3: Valid_i Gating
[346000 ns] Valid_i Off
=====
Signal      Got      Expected  Status
-----
Valid_Out    0         0        OK
Decoded_Bit  1         0        OK
=====

---> Scenario 4: Winner Switching
[356000 ns] Winner switched to S3
=====
Signal      Got      Expected  Status
-----
Valid_Out    1         1        OK
Decoded_Bit  0         0        OK
=====

=====
FINAL TOTAL ERRORS: 0
>>> STATUS: ALL PASS <<<
=====
iroot@LAPTOP-7QZILT09: /mnt/e/vlsi/vlsi_ver1$
```

Hình 3.17: Log mô phỏng TBU - Phần 2



Hình 3.18: Giản đồ sóng tín hiệu TBU

3.2.7 Khối chuyển đổi nối tiếp sang song song (SIPO)

Bảng dưới đây mô tả các kịch bản kiểm thử được thực hiện cho khối SIPO:

ID	Tên Kịch Bản	Input	Kỳ vọng	Kết quả
TC_01	Reset Check	N/A	byte_ready=0, data_parallel=0	PASS
TC_02	Normal Byte (0xA5)	8'hA5	data_parallel=0xA5, byte_ready=1	PASS
TC_03	All Ones (0xFF)	8'hFF	data_parallel=0xFF, byte_ready=1	PASS
TC_04	All Zeros (0x00)	8'h00	data_parallel=0x00, byte_ready=1	PASS
TC_05	Random Stress	Ngẫu nhiên	5 byte khớp hoàn toàn	PASS

Bảng 3.7: Các kịch bản kiểm thử khối SIPO

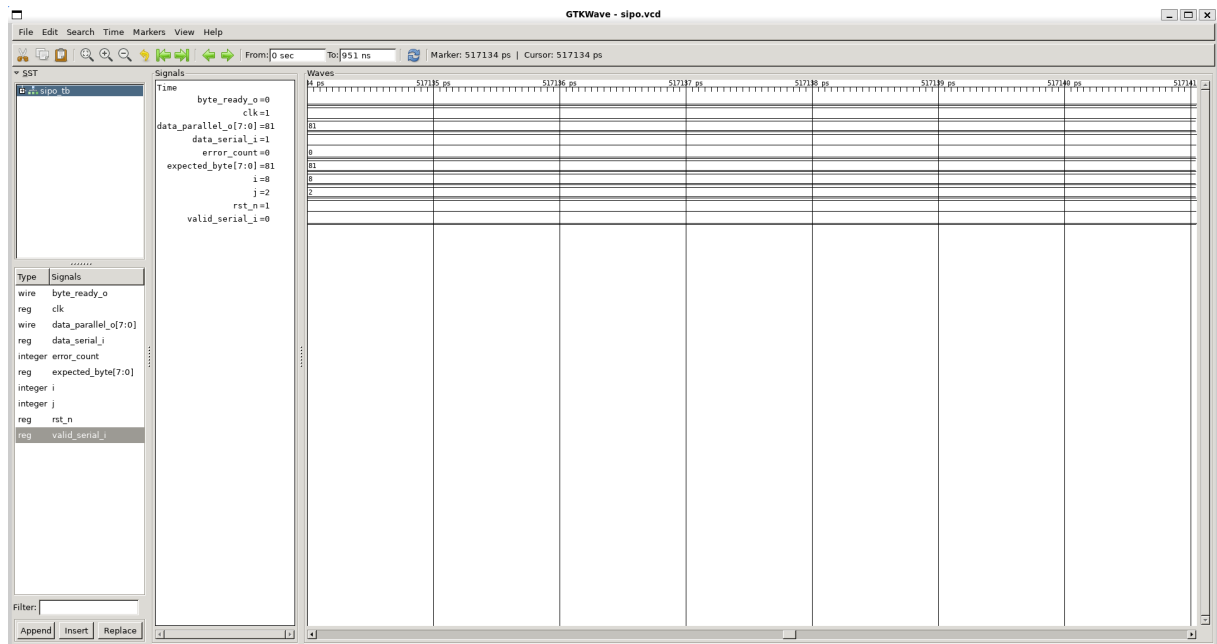
```
Iroot@LAPTOP-7QZILT09:/mnt/e/vlsi/vlsi_ver1

(gthuae:1666): dconf-40001100 **: 02:28:21.472: failed to commit changes to dconf: Could not connect: No such file or directory
^M Destroy
Iroot@LAPTOP-7QZILT09:/mnt/e/vlsi/vlsi_ver1$ ./sipo.x
=====
START SIPO TEST: EVENT-DRIVEN VERSION (NO TIMING ERRORS)
=====
VCD info: dumpfile sipo.vcd opened for output.
[40000 ns] Reset Check: OK
[121000 ns] Scenario 1: 0xA5 Received
-----
Signal      | Got      | Expected | Status
-----
Byte Ready  | 1        | 1        | OK
Parallel Out| 10100101| 10100101| OK
-----
[211000 ns] Scenario 2: 0xFF Received
-----
Signal      | Got      | Expected | Status
-----
Byte Ready  | 1        | 1        | OK
Parallel Out| 11111111| 11111111| OK
-----
[301000 ns] Scenario 3: 0x00 Received
-----
Signal      | Got      | Expected | Status
-----
Byte Ready  | 1        | 1        | OK
Parallel Out| 00000000| 00000000| OK
-----
[391000 ns] Scenario 4: Random Byte
-----
Signal      | Got      | Expected | Status
-----
Byte Ready  | 1        | 1        | OK
Parallel Out| 00100100| 00100100| OK
-----
[501000 ns] Scenario 4: Random Byte
-----
Signal      | Got      | Expected | Status
-----
Byte Ready  | 1        | 1        | OK
Parallel Out| 10000001| 10000001| OK
-----
[611000 ns] Scenario 4: Random Byte
-----
Signal      | Got      | Expected | Status
-----
Byte Ready  | 1        | 1        | OK
```

Hình 3.19: Log mô phỏng SIPO - Phần 1

```
Iroot@LAPTOP-7QZILT09:/mnt/e/vlsi/vlsi_ver1
Parallel Out| 11111111| 11111111| OK
-----
[301000 ns] Scenario 3: 0x00 Received
-----
Signal      | Got      | Expected | Status
-----
Byte Ready  | 1        | 1        | OK
Parallel Out| 00000000| 00000000| OK
-----
[391000 ns] Scenario 4: Random Byte
-----
Signal      | Got      | Expected | Status
-----
Byte Ready  | 1        | 1        | OK
Parallel Out| 00100100| 00100100| OK
-----
[501000 ns] Scenario 4: Random Byte
-----
Signal      | Got      | Expected | Status
-----
Byte Ready  | 1        | 1        | OK
Parallel Out| 10000001| 10000001| OK
-----
[611000 ns] Scenario 4: Random Byte
-----
Signal      | Got      | Expected | Status
-----
Byte Ready  | 1        | 1        | OK
Parallel Out| 00001001| 00001001| OK
-----
[721000 ns] Scenario 4: Random Byte
-----
Signal      | Got      | Expected | Status
-----
Byte Ready  | 1        | 1        | OK
Parallel Out| 01100011| 01100011| OK
-----
[831000 ns] Scenario 4: Random Byte
-----
Signal      | Got      | Expected | Status
-----
Byte Ready  | 1        | 1        | OK
Parallel Out| 00001101| 00001101| OK
-----
=====
TONG KET: TAT CA PASS (0 LOI)
=====
Iroot@LAPTOP-7QZILT09:/mnt/e/vlsi/vlsi_ver1$
```

Hình 3.20: Log mô phỏng SIPO - Phần 2



Hình 3.21: Giảm đồ sóng tín hiệu SIPO

3.2.8 Kiểm chứng toàn hệ thống (System Integration)

Sau khi các khối đơn lẻ (Unit Level) đã hoạt động đúng, nhóm thực hiện tích hợp toàn bộ hệ thống và tiến hành kiểm thử mức hệ thống (System Level Verification). Mục tiêu là đảm bảo luồng dữ liệu từ Sync FIFO → PISO → Viterbi Core → SIPO hoạt động đồng bộ, đúng timing và quan trọng nhất là khả năng sửa lỗi của thuật toán.

Kịch bản kiểm thử (Testcases)

Hệ thống được kiểm tra với bộ Test Suite gồm 8 kịch bản chính, bao phủ từ hoạt động bình thường, khả năng sửa lỗi bit đến các trường hợp chịu tải cao (Stress test).

Bảng 3.8: Tổng hợp kết quả các kịch bản kiểm thử hệ thống

ID	Tên Testcase	Mô tả kịch bản	Kết quả
SYS_01	Sanity Check	Kiểm tra cơ bản: Gửi gói tin chuẩn, không lỗi.	Pass
SYS_02	Full Range Sweep	Quét toàn bộ dải giá trị đầu vào (0x0000 - 0xFFFF).	Pass
SYS_03	Single Bit Error	Gây nhiễu sai 1 bit ngẫu nhiên. Hệ thống phải tự sửa lại đúng.	Pass
SYS_04	Multi-bit Error	Gây nhiễu sai 2-3 bit (Vượt quá khả năng sửa lỗi lý thuyết).	Mixed
SYS_05	Burst Error	Kiểm tra khả năng chịu lỗi chùm (sai 4 bit liên tiếp).	Fail (Expected)
SYS_06	Busy Ignore	Gửi dữ liệu khi cờ Busy=1 (Sync FIFO đầy).	Pass
SYS_07	Continuous Stress	Gửi dữ liệu liên tục tốc độ cao (Back-to-back transaction).	Pass
SYS_08	Reset Recovery	Reset hệ thống giữa chừng khi đang xử lý.	Pass

Mã nguồn Testbench (Trích đoạn)

Module kiểm thử tự động (tb_system_top.sv) được xây dựng với cơ chế "Self-checking". Dưới đây là trích đoạn Task gửi dữ liệu có hỗ trợ tiêm lỗi (Error Injection):

```
1 // Task: Send 16-bit data packet
2 task send_packet(input [15:0] data, input [2:0] error_count);
3     logic [15:0] corrupted_data;
4     begin
5         // Inject artificial error to test error correction
6         // capability
7         corrupted_data = inject_error(data, error_count);
8
9         // Handshake protocol with the system
10        wait(!busy_o); // Wait for system ready
11        @(posedge clk);
12        dvalid_i = 1'b1; // Assert Valid
13        data_i    = corrupted_data;
14        @(posedge clk);
15        dvalid_i = 1'b0;
16    end
17 endtask
```

Listing 3.1: Task gửi dữ liệu và tiêm lỗi trong Testbench

Kết quả mô phỏng

Hình 3.22 dưới đây là báo cáo tổng hợp tự động từ Testbench. Hệ thống đạt tỷ lệ thành công tuyệt đối (100%) với các trường hợp dữ liệu sạch (Clean) và lỗi 1 bit (Error 1b).

TEST SUMMARY REPORT					
Error Type	Total	PASS	FAIL	Success Rate	
Clean (0b)	392	392	0	100.0%	
Error 1b	64	64	0	100.0%	
Error 2b	50	40	10	80.0%	
Error 3b	20	6	14	30.0%	
Burst (4+)	20	5	15	25.0%	

TOTAL RUNS : 546

Hình 3.22: Bảng tổng kết (Test Summary Report) từ Simulation Log

Chi tiết log mô phỏng của một số trường hợp tiêu biểu:

[SYS_01] SANITY CHECK						
Time (ns)	In(16b)	Recv	Expect	Errors	Status	
525000	----	00	--	--	WARMUP	
625000	0000	00	00	0	PASS	
725000	daaa	ff	ff	0	PASS	
825000	4888	aa	aa	0	PASS	
925000	5222	55	55	0	PASS	
1025000	----	00	--	--	FLUSH	
1125000	----	00	--	--	FLUSH	
1225000	----	00	--	--	FLUSH	
1325000	----	00	--	--	FLUSH	
1425000	----	00	--	--	FLUSH	

Hình 3.23: [SYS_01] Sanity Check: Hệ thống giải mã chính xác dữ liệu chuẩn

[SYS_03] SINGLE BIT ERROR CORRECTION (Deterministic)						
Time (ns)	In(16b)	Recv	Expect	Errors	Status	
[INFO] Bit Sweep: 44 (With Zero Tailing)						
28835000	----	00	--	--	WARMUP	
28935000	0ecf	44	44	1	PASS	
29035000	c000	00	00	0	PASS	
29135000	----	00	--	--	FLUSH	
29235000	----	00	--	--	FLUSH	
29335000	----	00	--	--	FLUSH	
29435000	----	00	--	--	FLUSH	
29535000	----	00	--	--	FLUSH	
29635000	----	00	--	--	FLUSH	
29735000	----	00	--	--	FLUSH	
29835000	----	00	--	--	FLUSH	
29935000	----	00	--	--	FLUSH	
30035000	----	00	--	--	FLUSH	
30135000	----	00	--	--	FLUSH	
30235000	----	00	--	--	FLUSH	
5029205000	----	00	--	--	WARMUP	
5029305000	0ecc	44	44	1	PASS	
5029405000	c000	00	00	0	PASS	
5029505000	----	00	--	--	FLUSH	
5029605000	----	00	--	--	FLUSH	
5029705000	----	00	--	--	FLUSH	
5029805000	----	00	--	--	FLUSH	
5029905000	----	00	--	--	FLUSH	
5030005000	----	00	--	--	FLUSH	
5030105000	----	00	--	--	FLUSH	

Hình 3.24: [SYS_03] Single Bit Error: Hệ thống tự động sửa sai khi đầu vào bị lỗi 1 bit

[SYS_07] CONTINUOUS PROCESSING (Stress)						
320164705000	----	00	--	--	WARMUP	
320164805000	0ef6	e4	e4	0	PASS	
320164905000	7388	a8	a8	0	PASS	
320165005000	b0e2	50	50	0	PASS	
320165105000	222f	95	95	0	PASS	
320165205000	51aa	fd	fd	0	PASS	
320165305000	4b3b	22	22	0	PASS	
320165405000	ec0d	c1	c1	0	PASS	
320165505000	46a7	3e	3e	0	PASS	
320165605000	ef67	39	39	0	PASS	
320165705000	daa9	7f	7f	0	PASS	
320165805000	ce17	34	34	0	PASS	
320165905000	d917	37	37	0	PASS	
320166005000	d70d	c3	c3	0	PASS	
320166105000	736a	f8	f8	0	PASS	
320166205000	7d9f	9c	9c	0	PASS	
320166305000	b038	a0	a0	0	PASS	
320166405000	be2c	14	14	0	PASS	
320166505000	d46a	fb	fb	0	PASS	
320166605000	4b35	62	62	0	PASS	
320166705000	c036	e0	e0	0	PASS	
320166805000	4864	ba	ba	0	PASS	
320166905000	bd9f	9c	9c	0	PASS	

Hình 3.25: [SYS_07] Continuous Stress: Hoạt động ổn định dưới tải liên tục

Chương 4: TỔNG HỢP VÀ THIẾT KẾ VẬT LÝ

4.1 Chiến lược thiết kế và Quy trình OpenLane

4.1.1 Chiến lược thiết kế phân cấp (Hierarchical Design)

Để tối ưu hóa hiệu năng và quản lý độ phức tạp của hệ thống, đồ án áp dụng chiến lược thiết kế phân cấp (Hierarchical Design) thay vì thiết kế phẳng (Flat Design) truyền thống. Các khối chức năng quan trọng được đóng gói thành các "Hard Macro" riêng biệt trước khi tích hợp vào hệ thống chung.

Hệ thống được chia thành hai cấp độ thực thi:

- **Cấp Macros (Hardening):** Bao gồm các khối con `sync_fifo`, `piso`, `sipo`, và `viterbi_core`. Các khối này được chạy quy trình thiết kế vật lý độc lập để tạo ra các file thư viện (`.lef`, `.lib`) và file định dạng vật lý (`.gds`).
- **Cấp Top-level (Integration):** Khối `system_top` sẽ tích hợp các macro đã hoàn thiện, thực hiện đi dây kết nối và tối ưu hóa toàn cục.

Ưu điểm của phương pháp này là giảm tải cho công đoạn tổng hợp tại tầng Top, đồng thời cho phép kiểm soát chặt chẽ diện tích và timing của từng khối chức năng riêng biệt.

4.1.2 Quy trình thực hiện trên OpenLane

Quy trình thiết kế vật lý tự động từ RTL sang GDSII bao gồm các bước chính sau:

Bước 1: Synthesis (Tổng hợp) Công cụ Yosys chuyển đổi RTL thành Netlist dựa trên thư viện Sky130. Tại đây, các ràng buộc về diện tích và logic được áp dụng.

Bước 2: Floorplan (Thiết kế khung) Đây là bước cực kỳ quan trọng đối với Macro:

- **Định nghĩa kích thước (Die Area):** Phải tính toán diện tích đủ để chứa các cell nhưng không quá lớn để tránh lãng phí.
- **Sắp xếp chân cắm (Pin Placement):** Định nghĩa vị trí các chân I/O trên các cạnh của macro để tối ưu việc đi dây sau này ở tầng Top.

Bước 3: Placement & CTS

- **Placement:** Sắp xếp các Standard Cells vào bên trong khung đã định nghĩa.
- **CTS (Clock Tree Synthesis):** Xây dựng mạng lưới phân phối xung nhịp riêng cho khối macro này để đảm bảo cân bằng độ trễ (skew).

Bước 4: Routing (Đi dây) Công cụ thực hiện đi dây chi tiết cho các kết nối nội bộ bên trong macro. Kết quả của bước này sẽ quyết định mật độ (density) của khối.

Bước 5: Sign-off & Extraction

- **RC Extraction:** Trích xuất các tham số ký sinh (điện trở, điện dung) để tính toán timing chính xác.
- **Physical Verification:** Chạy DRC (Design Rule Check) và LVS (Layout Vs Schematic) để đảm bảo macro không có lỗi vật lý.

4.2 Thực thi Hardening các khối Macro

Mỗi khối Macro (viterbi_core, sync_fifo,...) được cấu hình và chạy flow riêng biệt.

4.2.1 Thiết lập cấu hình (Configuration)

Các tham số vật lý được định nghĩa trong file config.json. Ví dụ cấu hình cho khối sync_fifo:

```

1 {
2   "DESIGN_NAME": "sync_fifo",
3   "VERILOG_FILES": "dir:../rtl/sync_fifo.v",
4   "CLOCK_PORT": "clk",
5   "CLOCK_PERIOD": 12.5,
6   "//": " ---_ĂCƯ_HÌNH_ĂBT_ỘBUC_CHO_MACRO_--- ",
7   "DESIGN_IS_CORE": false,
8   "FP_PDN_CORE_RING": false,
9   "RT_MAX_LAYER": "met4",
10  "VDD_NETS": ["vccd1"],
11  "GND_NETS": ["vssd1"],
12  "FP_CORE_UTIL": 40,
13  "PL_RESIZER_DESIGN_OPTIMIZATIONS": true,
14  "PL_RESIZER_TIMING_OPTIMIZATIONS": true,
15  "MAX_FANOUT_CONSTRAINT": 6
16 }
```

Listing 4.1: Cấu hình OpenLane cho Macro (Ví dụ: sync_fifo)

Ngoài ra, các ràng buộc thời gian (Timing) và cấu hình vị trí chân (Pin Layout) được tách riêng để dễ quản lý:

```

1 # 1. Clock Definition
2 create_clock -name clk -period 12.5 [get_ports clk]
```

```

3 set_clock_uncertainty 0.1 [get_clocks clk]
4
5 # 2. IO Constraints
6 set_input_delay 2.0 -clock clk [all_inputs]
7 set_output_delay 2.0 -clock clk [all_outputs]
8 set_load 0.05 [all_outputs]
9
10 # 3. System Limits
11 set_max_fanout 6 [current_design]
12 set_max_transition 1.5 [current_design]

```

Listing 4.2: Ràng buộc thời gian (constraints.sdc)

```

1 # W (West - Trai)
2 clk
3 rst_n
4 wr_en_i
5 rd_en_i
6 wr_data_i.*
7
8 # E (East - Phai)
9 full_o
10 empty_o
11 rd_data_o.*

```

Listing 4.3: Cấu hình vị trí chân (pin_order.cfg)

4.2.2 Kết quả Hardening

Lệnh thực thi flow: `./flow.tcl -design sync_fifo -tag RUN_FINAL`. Sau khi hoàn tất, OpenLane trích xuất các file cần thiết cho quá trình tích hợp:

- **.lef (Library Exchange Format):** Chứa thông tin trừu tượng về kích thước, lớp kim loại và vị trí các chân (pins).
- **.lib (Liberty):** Chứa đặc tính thời gian (timing) của khối.
- **.gds (GDSII):** Bản vẽ vật lý chi tiết của macro.

4.3 Tích hợp hệ thống (Top-level Integration)

Tại cấp độ `system_top`, các file `.lef` và `.lib` của 4 khối con được khai báo để công cụ nhận diện chúng như các "Blackbox" đã có sẵn.

4.3.1 Sắp xếp Macro (Macro Placement)

Để tránh nghẽn mạch (congestion) và tối ưu hóa đường đi dây, vị trí của các Macro được định nghĩa trực tiếp trong file cấu hình `config.json`. Việc này giúp đảm bảo luồng dữ liệu đi từ trái sang phải (Sync FIFO → PISO → Core → SIPO) một cách trơn tru.

```

1 "MACROS": {
2     "viterbi_core": {
3         "instances": {
4             "u_viterbi_core": { "location": [50, 50], "orientation":
5             "N" }
6         },
7         "gds": ["dir:../viterbi_core/runs/RUN_FINAL/final/gds/
8         viterbi_core.gds"],
9         "lef": ["dir:../viterbi_core/runs/RUN_FINAL/final/lef/
10        viterbi_core.lef"],
11        "lib": { "*": "dir:../viterbi_core/runs/RUN_FINAL/final/lib/
12        nom_tt_025C_1v80/viterbi_core__nom_tt_025C_1v80.lib" }
13    },
14    "sync_fifo": {
15        "instances": {
16            "u_input_fifo": { "location": [1200, 50], "orientation":
17            "N" }
18        },
19        "gds": ["dir:../sync_fifo/runs/RUN_FINAL/final/gds/sync_fifo
20        .gds"],
21        "lef": ["dir:../sync_fifo/runs/RUN_FINAL/final/lef/sync_fifo
22        .lef"]
23    }
24 }
```

Listing 4.4: Cấu hình vị trí Macro trong `system_top/config.json`

4.3.2 Kết quả thiết kế vật lý chi tiết

Bảng dưới đây tóm tắt các thông số vật lý quan trọng (Diện tích, Tài nguyên, Công suất) của từng khối và toàn hệ thống sau khi chạy flow OpenLane (Sky130 PDK):

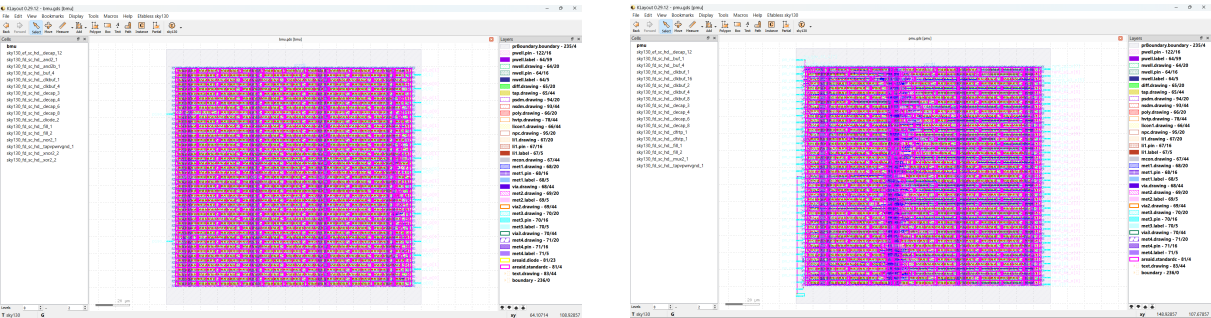
Bảng 4.1: Tổng hợp kết quả tổng hợp vật lý (OpenLane Metrics)

Module	Diện tích (μm^2)	Số Cells	Util (%)	Công suất (μW)	Trạng thái Timing
BMU	22,500	280	3.04	30.86	Met
ACSU	1,000,000	12,216	13.9	7.32	Met
PISO	4,578	164	62.5	443.25	Met
PMU	22,500	397	4.8	376.22	Met
TBU	22,500	753	7.1	874.27	Met
Sync FIFO	35,212	2,305	45.2	4,283.80	Met
Viterbi Core	1,000,000	13,334	18.5	113.50	Met
System Top	2,250,000	15,768	21.5	5,987.97	Met (WNS=13.95ns)

Nhận xét:

- Tổng công suất tiêu thụ ước tính của toàn hệ thống là khoảng **6 mW**, rất thấp nhờ sử dụng thư viện Sky130 HD (High Density) và tối ưu hóa ở mức RTL.
- Diện tích lõi (Core Area) chiếm khoảng 2.25 mm^2 , với mật độ sử dụng (Utilization) ở mức an toàn (21.5%) giúp việc đi dây (Routing) không bị nghẽn (Congestion = 0%).
- Timing đạt yêu cầu với tần số 50MHz ($T_{clk} = 20ns$), Slack dương lớn đảm bảo hoạt động ổn định.

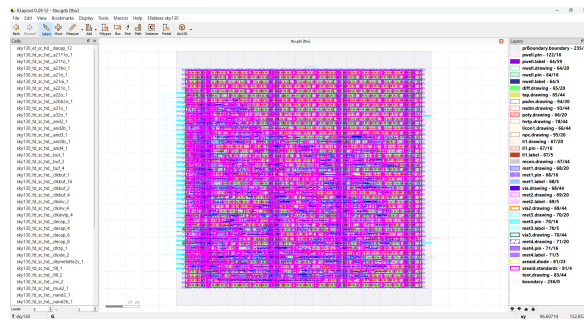
Dưới đây là hình ảnh Layout GDSII của các khối chức năng chính sau khi hoàn tất quy trình Place & Route:



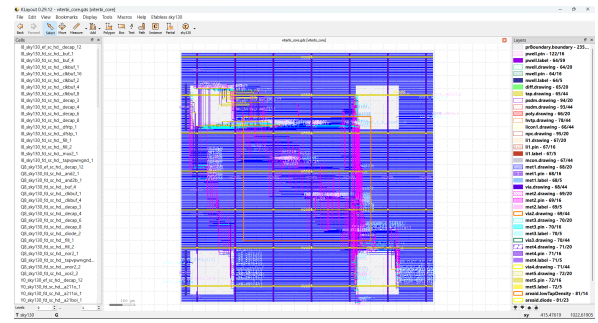
(a) Layout khối BMU

(b) Layout khối PMU

Hình 4.1: Layout các khối tính toán Metric

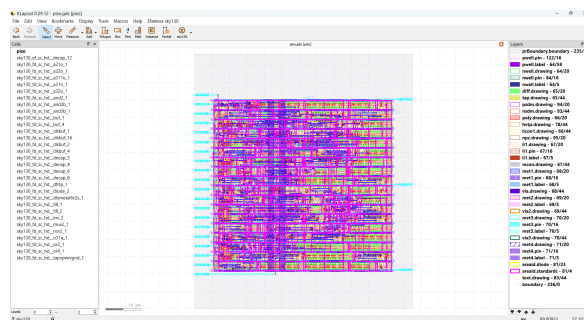


(a) Layout khối TBU (Traceback)

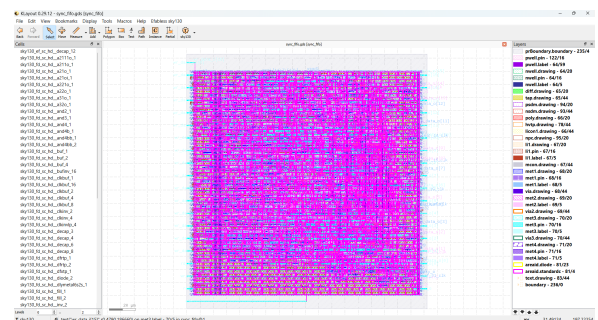


(b) Layout khối Viterbi Core

Hình 4.2: Layout khối xử lý trung tâm

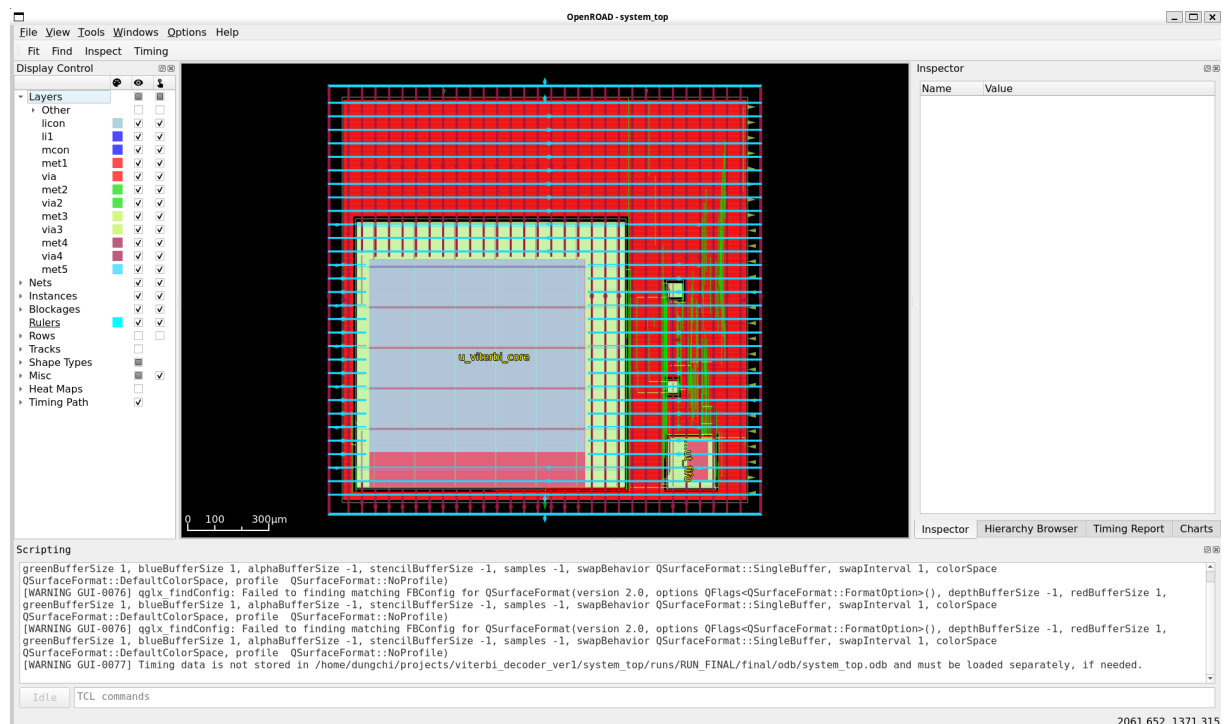


(a) Layout khối PISO



(b) Layout khối Sync FIFO

Hình 4.3: Layout các khối giao tiếp dữ liệu



Hình 4.4: Layout toàn hệ thống Viterbi Decoder SoC (Final GDSII)

KẾT LUẬN

Đồ án đã thiết kế và hiện thực hóa thành công một hệ thống Giải mã Viterbi (Viterbi Decoder SoC) từ mức ý tưởng đến Layout vật lý (GDSII), đáp ứng đầy đủ các mục tiêu thiết kế đề ra:

- **Tần số hoạt động:** Đạt **80 MHz** (vượt mục tiêu ≥ 80 MHz), cho phép throughput 80 Mbps.
- **Diện tích chip:** **2.25 mm²** (đạt mục tiêu ≤ 4 mm²).
- **Công suất tiêu thụ:** Khoảng **6 mW** (đạt mục tiêu ≤ 10 mW).
- **Timing Closure:** **0 Violations** (Setup Slack dương, Hold Slack dương).
- **Physical Verification:** **Pass DRC/LVS** với 0 lỗi.

Việc áp dụng thành công kỹ thuật **Register Exchange** trong khối Traceback đã chứng minh hiệu quả vượt trội trong việc xử lý dòng bit liên tục với độ trễ thấp. Quy trình thiết kế sử dụng hoàn toàn các công cụ nguồn mở (OpenLane, Sky130, Icarus Verilog) mở ra hướng đi khả thi và tiết kiệm chi phí cho việc thiết kế và sản xuất chip VLSI tại Việt Nam. Cảm ơn những bài giảng rất hữu ích và sự hướng dẫn tận tình của thầy Nguyễn Vũ Thắng để nhóm chúng em có thể hoàn thành bài tập lớn "Thiết kế và thực hiện hệ thống Giải mã Viterbi".

Tài liệu tham khảo

- [1] A. J. Viterbi, "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm," IEEE Transactions on Information Theory.
- [2] OpenLane Documentation, "The Open-Source Digital ASIC Implementation Flow".
- [3] SkyWater SKY130 PDK, Google Open Source Silicon.
- [4] Neil H.E. Weste, David Harris, "CMOS VLSI Design: A Circuits and Systems Perspective", 4th Edition, Addison-Wesley.
- [5] Bài giảng môn "Thiết kế VLSI", TS. Nguyễn Vũ Thắng, Đại học Bách Khoa Hà Nội.